Accélération sur GPU pour la reconstruction en tomographie à rayons X

Nicolas GAC

Maître de conférences à l'université Paris Saclay
Laboratoire des Signaux et Systèmes (L2S) - Groupe Problèmes Inverses (GPI)

M2 ATSI - UE Imagerie médicale

1. Adéquation Algorithme Architecture (A^3)
   - Problématique et positionnement
   - A^3 au sein du GPI
2. Calcul intensif sur GPU/FPGA
   - Paire de projection/rétroprojection $\mathcal{P}/\mathcal{R}$
3. GPU (Graphic Processing Units) : hardware and software
   - GPU (re)designed as a many core architecure
   - Programming in CUDA
   - A toy example : acceleration of matrix multiplication

1. Adéquation Algorithme Architecture (A^3)
   - Problématique et positionnement
   - A^3 au sein du GPI
2. Calcul intensif sur GPU/FPGA
   - Paire de projection/rétroprojection $\mathcal{P}/\mathcal{R}$
3. GPU (Graphic Processing Units) : hardware and software
   - GPU (re)designed as a many core architecure
   - Programming in CUDA
   - A toy example : acceleration of matrix multiplication

Adéquation Algorithme Architecture *[thème C du GDR ISIS]*

Meilleure **combinaison d'architectures et d'algorithmes**

sous **contraintes** de :

- temps de traitement
- précision de calcul
- consommation énergétique

Adéquation Algorithme Architecture *[thème C du GDR ISIS]*

Meilleure **combinaison d'architectures et d'algorithmes**

sous **contraintes** de :

- temps de traitement
- précision de calcul
- consommation énergétique
- coût du système de calcul
- temps de développement

Adéquation Algorithme Architecture *[thème C du GDR ISIS]*

Meilleure **combinaison d'architectures et d'algorithmes**

sous **contraintes** de :

- temps de traitement
- précision de calcul
- consommation énergétique
- coût du système de calcul
- temps de développement

Différentes approches A^3

- outils pour le TdSI (ex : Syndex, PREESM...)
- proche des capteurs (*smart sensors*)
- proche de l'application et des architectures

Adéquation Algorithme Architecture *[thème C du GDR ISIS]*

Meilleure **combinaison d'architectures et d'algorithmes**

sous **contraintes** de :

- temps de traitement
- précision de calcul
- consommation énergétique
- coût du système de calcul
- temps de développement

Différentes approches A^3

- outils pour le TdSI (ex : Syndex, PREESM...)
- proche des capteurs (*smart sensors*)
- **proche de l'application et des architectures**
  - contextes applicatifs **spécifiques avec contraintes fortes**
  - **solutions « sur mesure »** pour des performances optimales.

Adéquation Algorithme Architecture

Enjeux

- **Interaction humaine** la plus forte possible entre expertises :
  - **applicative**
  - **algorithmique**
  - **matérielle**
- Prise en compte des **évolutions technologiques**
- Difficile mais impérative **généralisation de la démarche A^3**

Périmètre applicatif

Problème inverse

Reconstruction à partir des mesures d'un instrument

Verrous

#1 Problèmes dit « mal posé »

#2 Problème en grande dimension

Périmètre applicatif

Problème inverse

Reconstruction à partir des mesures d'un instrument

Verrous

#1 Problèmes dit « mal posé »

#2 Problème en grande dimension

Méthodes analytiques

Modélisation souvent réductrice

☺ rapide ☹ moins robuste

Périmètre applicatif

Problème inverse

Reconstruction à partir des mesures d'un instrument

Verrous

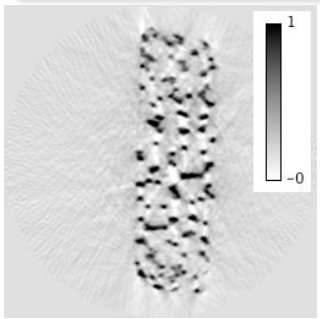
#1 Problèmes dit « mal posé »

#2 Problème en grande dimension

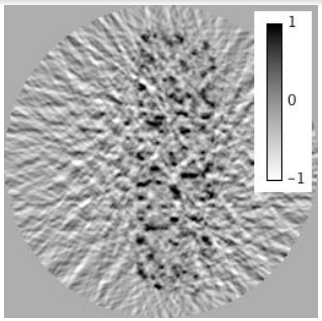
Méthodes analytiques

Modélisation souvent réductrice

☺ rapide ☹ moins robuste



FDK
(256/256 proj.)



FDK
(32/256 proj.)

Reconstructions d'une mousse métallique [C_{2011a}]

Périmètre applicatif

Problème inverse

Reconstruction à partir des mesures d'un instrument

Verrous

#1 Problèmes dit « mal posé »

#2 Problème en grande dimension

Méthodes analytiques

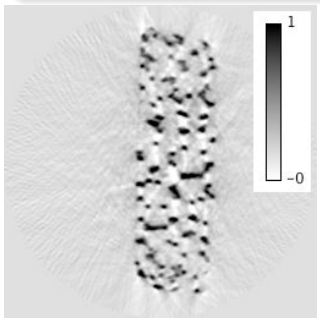
Modélisation souvent réductrice

☺ rapide ☹ moins robuste

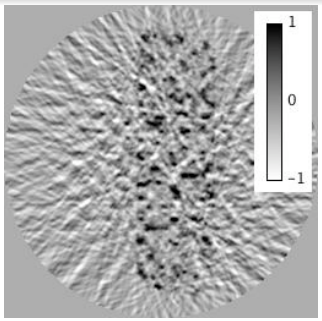
Méthodes itératives par optimisation d'un critère

Modélisation plus riche

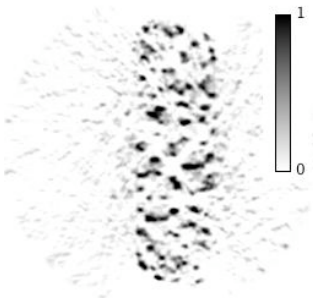
☺ plus robuste ☹ lent



FDK
(256/256 proj.)



FDK
(32/256 proj.)



Itératif
(32/256 proj.)

Reconstructions d'une mousse métallique [C_{2011a}]

Larges champs d'exploration algorithmique

$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Larges champs d'exploration algorithmique

$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
4. Choix du modèle de l'objet $\mathbb{M}_f$
5. Choix de l'algorithme d'optimisation

Larges champs d'exploration algorithmique

$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. **Choix de l'estimateur**
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
4. Choix du modèle de l'objet $\mathbb{M}_f$
5. Choix de l'algorithme d'optimisation

Larges champs d'exploration algorithmique

$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. **Choix de l'estimateur**
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
4. Choix du modèle de l'objet $\mathbb{M}_f$
5. Choix de l'algorithme d'optimisation

Larges champs d'exploration algorithmique

$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. **Choix du modèle de l'instrument $\boldsymbol{H}$**
3. Choix du modèle des incertitudes ϵ
   - *Lois normale, de student-t* [C$_{2017d}$] ...
   - *Error splitting model* [R$_{2019b}$]
4. Choix du modèle de l'objet $\mathbb{M}_f$
5. Choix de l'algorithme d'optimisation

Larges champs d'exploration algorithmique

$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. **Choix du modèle des incertitudes ϵ**
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
5. Choix de l'algorithme d'optimisation

Larges champs d'exploration algorithmique

$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \boldsymbol{\epsilon}$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
$\boldsymbol{\epsilon}$: incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes $\boldsymbol{\epsilon}$
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. **Choix du modèle de l'objet** $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. Choix de l'algorithme d'optimisation

Larges champs d'exploration algorithmique

$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \boldsymbol{\epsilon}$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
$\boldsymbol{\epsilon}$: incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes $\boldsymbol{\epsilon}$
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. **Choix de l'algorithme d'optimisation**
   - *Descente de gradient simple ou conjuguée*
   - *ADMM, split Bergman...*

Larges champs d'exploration algorithmique

$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Moindre carré reg. quadratique

$$J(\boldsymbol{f}) = \|\boldsymbol{g} - \boldsymbol{H}\boldsymbol{f}\|^2 + \lambda\|\boldsymbol{D}\boldsymbol{f}\|^2$$

$$\boldsymbol{f}^{(n+1)} = \boldsymbol{f}^{(n)} - \alpha\nabla J(\boldsymbol{f}^{(n)})$$

Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. Choix de l'algorithme d'optimisation
   - *Descente de gradient simple ou conjuguée*
   - *ADMM, split Bergman...*

Larges champs d'exploration algorithmique

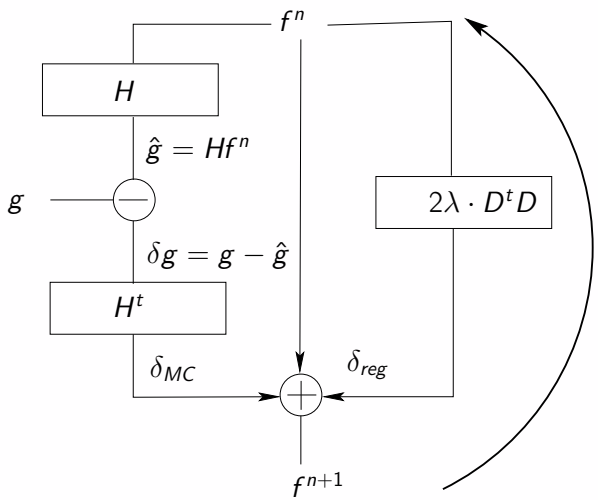
$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Moindre carré reg. quadratique

$$J(\boldsymbol{f}) = \|\boldsymbol{g} - \boldsymbol{H}\boldsymbol{f}\|^2 + \lambda\|\boldsymbol{D}\boldsymbol{f}\|^2$$

$$\boldsymbol{f}^{(n+1)} = \boldsymbol{f}^{(n)} - \alpha\nabla J(\boldsymbol{f}^{(n)})$$



Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. Choix de l'algorithme d'optimisation
   - *Descente de gradient simple ou conjuguée*
   - *ADMM, split Bergman...*

Larges champs d'exploration algorithmique

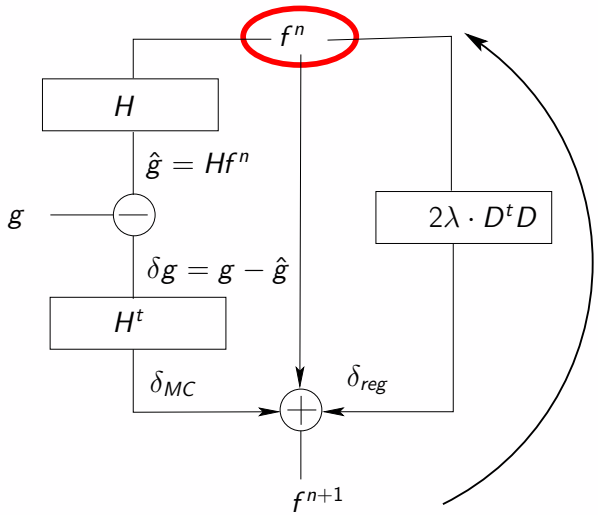
$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Moindre carré reg. quadratique

$$J(\boldsymbol{f}) = \|\boldsymbol{g} - \boldsymbol{H}\boldsymbol{f}\|^2 + \lambda\|\boldsymbol{D}\boldsymbol{f}\|^2$$

$$\boldsymbol{f}^{(n+1)} = \boldsymbol{f}^{(n)} - \alpha\nabla J(\boldsymbol{f}^{(n)})$$



Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. Choix de l'algorithme d'optimisation
   - *Descente de gradient simple ou conjuguée*
   - *ADMM, split Bergman...*

Larges champs d'exploration algorithmique

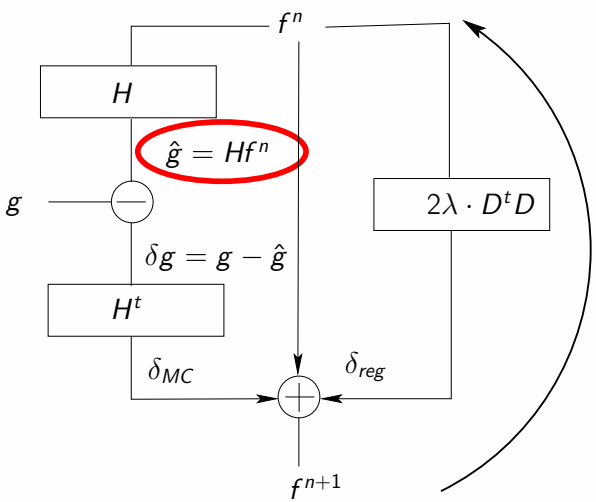
$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Moindre carré reg. quadratique

$$J(\boldsymbol{f}) = \|\boldsymbol{g} - \boldsymbol{H}\boldsymbol{f}\|^2 + \lambda\|\boldsymbol{D}\boldsymbol{f}\|^2$$

$$\boldsymbol{f}^{(n+1)} = \boldsymbol{f}^{(n)} - \alpha\nabla J(\boldsymbol{f}^{(n)})$$



Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. Choix de l'algorithme d'optimisation
   - *Descente de gradient simple ou conjuguée*
   - *ADMM, split Bergman...*

Larges champs d'exploration algorithmique

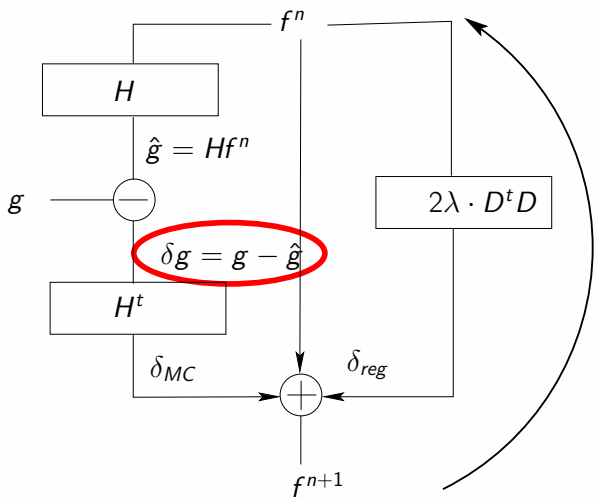
$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Moindre carré reg. quadratique

$$J(\boldsymbol{f}) = \|\boldsymbol{g} - \boldsymbol{H}\boldsymbol{f}\|^2 + \lambda\|\boldsymbol{D}\boldsymbol{f}\|^2$$

$$\boldsymbol{f}^{(n+1)} = \boldsymbol{f}^{(n)} - \alpha\nabla J(\boldsymbol{f}^{(n)})$$



Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. Choix de l'algorithme d'optimisation
   - *Descente de gradient simple ou conjuguée*
   - *ADMM, split Bergman...*

Larges champs d'exploration algorithmique

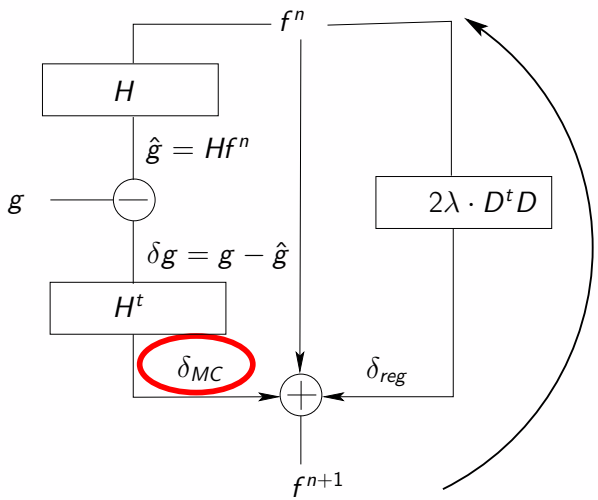
$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Moindre carré reg. quadratique

$$J(\boldsymbol{f}) = \|\boldsymbol{g} - \boldsymbol{H}\boldsymbol{f}\|^2 + \lambda\|\boldsymbol{D}\boldsymbol{f}\|^2$$

$$\boldsymbol{f}^{(n+1)} = \boldsymbol{f}^{(n)} - \alpha\nabla J(\boldsymbol{f}^{(n)})$$



Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. Choix de l'algorithme d'optimisation
   - *Descente de gradient simple ou conjuguée*
   - *ADMM, split Bergman...*

Larges champs d'exploration algorithmique

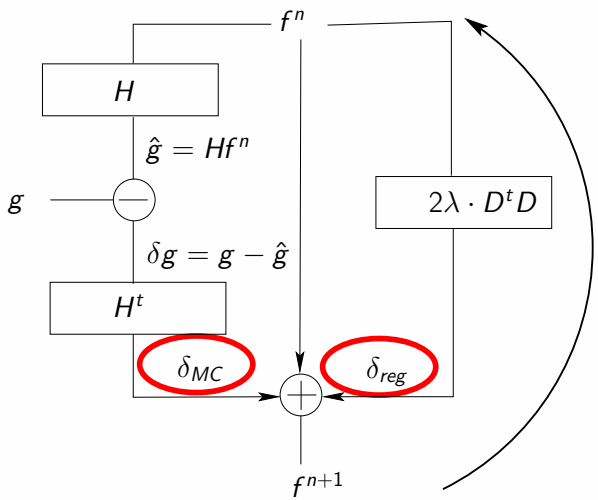
$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Moindre carré reg. quadratique

$$J(\boldsymbol{f}) = \|\boldsymbol{g} - \boldsymbol{H}\boldsymbol{f}\|^2 + \lambda\|\boldsymbol{D}\boldsymbol{f}\|^2$$

$$\boldsymbol{f}^{(n+1)} = \boldsymbol{f}^{(n)} - \alpha\nabla J(\boldsymbol{f}^{(n)})$$



Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. Choix de l'algorithme d'optimisation
   - *Descente de gradient simple ou conjuguée*
   - *ADMM, split Bergman...*

Larges champs d'exploration algorithmique

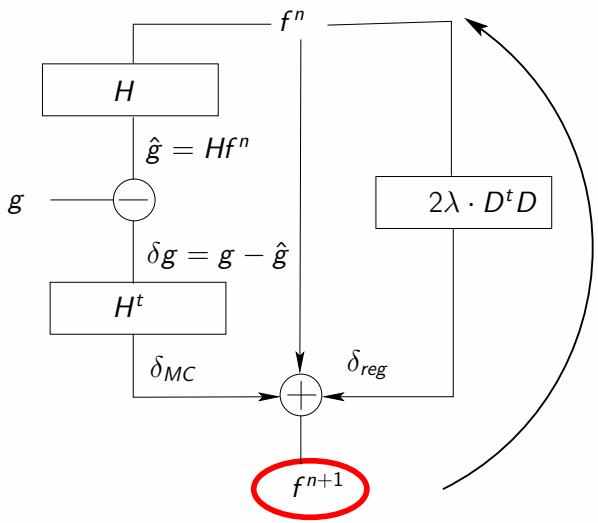
$\mathbb{M}_D : \boldsymbol{g} = \boldsymbol{H}\boldsymbol{f} + \epsilon$

$\boldsymbol{g}$: mesures de l'instrument
$\boldsymbol{H}$: modèle d'acquisition
$\boldsymbol{f}$: objet à reconstruire
ϵ : incertitudes sur $\boldsymbol{g}$ et $\boldsymbol{H}$

Moindre carré reg. quadratique

$$J(\boldsymbol{f}) = \|\boldsymbol{g} - \boldsymbol{H}\boldsymbol{f}\|^2 + \lambda\|\boldsymbol{D}\boldsymbol{f}\|^2$$

$$\boldsymbol{f}^{(n+1)} = \boldsymbol{f}^{(n)} - \alpha\nabla J(\boldsymbol{f}^{(n)})$$



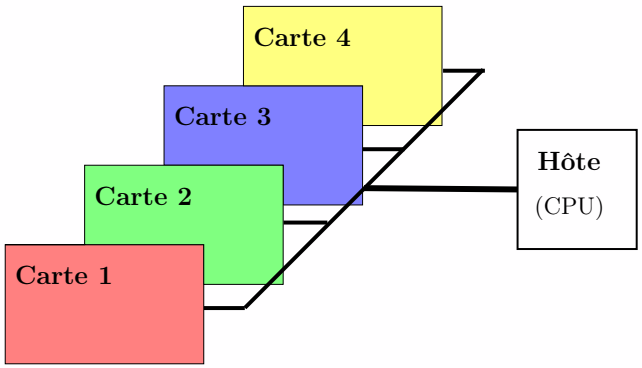
Approches bayésiennes

Loi a posteriori : $p(\boldsymbol{f}|\boldsymbol{g}) = \dfrac{p(\boldsymbol{g}|\boldsymbol{f};\mathbb{M}_D)p(\boldsymbol{f};\mathbb{M}_f)}{p(\boldsymbol{g})}$

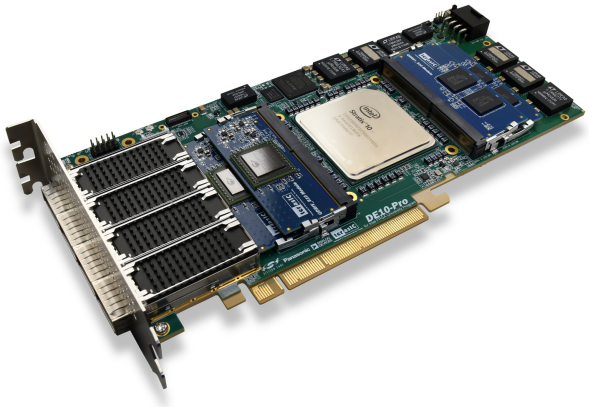
1. Choix de l'estimateur
   - MAP
   - EAP
2. Choix du modèle de l'instrument $\boldsymbol{H}$
3. Choix du modèle des incertitudes ϵ
   - *Lois normale, de student-t* [C_{2017d}] ...
   - *Error splitting model* [R_{2019b}]
4. Choix du modèle de l'objet $\mathbb{M}_f$
   - *Modèle de Gauss Markov Potts* [R_{2017b}]
   - *Représentation par ondelettes* [R_{2018b}]
5. Choix de l'algorithme d'optimisation
   - *Descente de gradient simple ou conjuguée*
   - *ADMM, split Bergman...*

Architectures de calcul étudiées

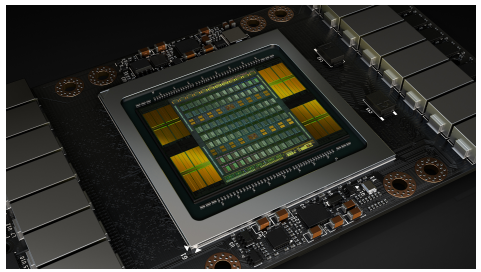
Plateforme de calcul avec cartes accélératrices



(a) plateforme de calcul



(b) carte PCIe



(c) puce de calcul

Puces accélératrices de calcul

- **GPU** : processeur massivement parallèle
- **FPGA** : conception d'une architecture sur mesure

$\Longrightarrow$ *Adaptées aux systèmes embarqués, PC ou supercalculateur HPC*

Larges champs d'exploration architecturale

GPU : algorithmes massivement parallèles

- **Parallélisation**
  - #1 à grains fins sur les [100 - 5000] coeurs de calcul
  - #2 à grains épais sur les [1-16] cartes accélératrices
- **Murs mémoire pour l'accès aux données**
  - #1 coeurs de calcul ⟷ mémoire DDR [1 - 32] Go
  - #2 cartes accélératrices ⟷ mémoire hôte [0.1 - 1] To
- **Boîte à outils d'accélération** (texture 2D/3D, tensor cores, *half float*...)

Larges champs d'exploration architecturale

GPU : algorithmes massivement parallèles

- **Parallélisation**
  - #1 à grains fins sur les [100 - 5000] coeurs de calcul
  - #2 à grains épais sur les [1-16] cartes accélératrices
- **Murs mémoire pour l'accès aux données**
  - #1 coeurs de calcul ⟷ mémoire DDR [1 - 32] Go
  - #2 cartes accélératrices ⟷ mémoire hôte [0.1 - 1] To
- **Boîte à outils d'accélération** (texture 2D/3D, tensor cores, *half float*...)

FPGA : outils de synthèse de haut niveau (HLS)

- **Architecture alternative au GPU**
  - #1 Architecture basse consommation
  - #2 Parallélisme de pipeline
- **Guide/méthodologie d'utilisation des outils HLS**

1 Adéquation Algorithme Architecture (A^3)
- Problématique et positionnement
- A^3 au sein du GPI

2 Calcul intensif sur GPU/FPGA
- Paire de projection/rétroprojection $\mathcal{P}/\mathcal{R}$

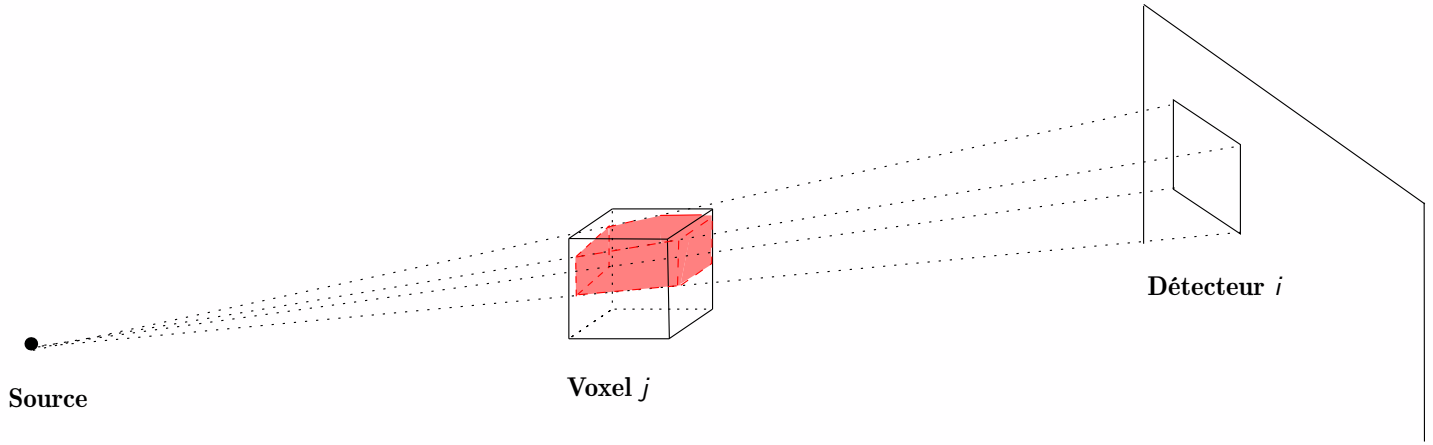
3 GPU (Graphic Processing Units) : hardware and software
- GPU (re)designed as a many core architecure
- Programming in CUDA
- A toy example : acceleration of matrix multiplication

Opérateurs $\mathcal{P}$ de projection et $\mathcal{R}$ de rétroprojection

Algorithme itératif

$$\boldsymbol{f}^{(n+1)} = \boldsymbol{f}^{(n)} + \alpha\, \boldsymbol{H}^t(\boldsymbol{g} - \boldsymbol{H}\boldsymbol{f}) + \ldots$$

$\boldsymbol{H}_{ij}$: contribution du voxel j dans le détecteur i



$\boldsymbol{H}$ de grande dimension

$\boldsymbol{f}$ *de taille* 2048^3 $\Longrightarrow$ $\boldsymbol{H}$ *de taille 256 Exa octets*

$\Longrightarrow$ Calcul de $\boldsymbol{H}$ et de $\boldsymbol{H}^t$ par une paire $\mathcal{P}/\mathcal{R}$

Démocratisation rapide des accélérateurs de type GPU

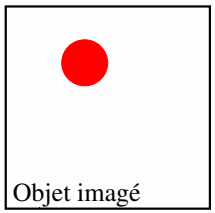
Cartes accélératrices utilisées dans les travaux publiés à la conférence Fully 3D spécialisée en reconstruction tomographique

- 2007 : 1er Workshop HPIR (High Performance Image Reconstruction)
- 2011 : Le mot clef Multi GPU apparait

	2003	2005	2007	2009	2011	2013	2015	2017	2019
GPU			**10**	**17**	**18**	**9**	**13**	**20**	**30**
Nvidia			10	14	17	7	10	20	
AMD				1	1		1		
Larabee/Xeon phi				2		2	2		
multi-GPU					3	3	3		1
CNN								2	13
Autres	**2**	**3**	**10**	**7**	**3**	**2**	**1**	**2**	
CPU	2	3	5	6	3	2	1	2	
Cell (IBM)			3						
DSP			2	1					
FPGA			**4**		**1**		**1**		

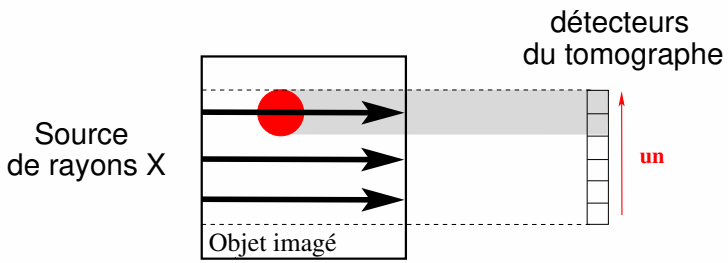
$\mathcal{P}/\mathcal{R}$: boucles naturelles de parallélisation

Opérateur de projection $\mathcal{P}$



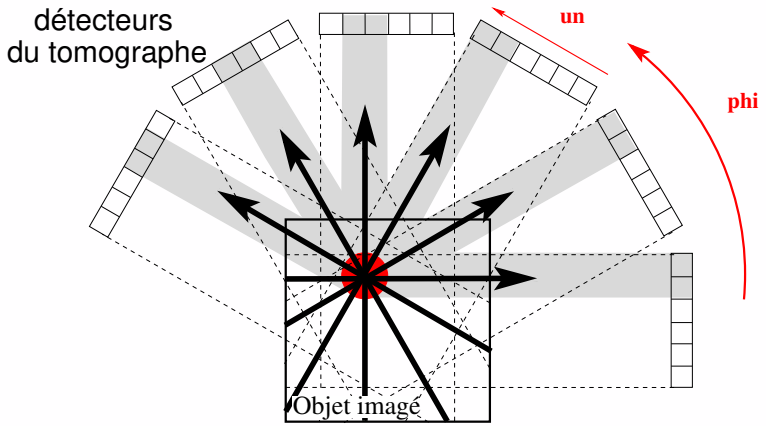
$\mathcal{P}/\mathcal{R}$: boucles naturelles de parallélisation

Opérateur de projection $\mathcal{P}$



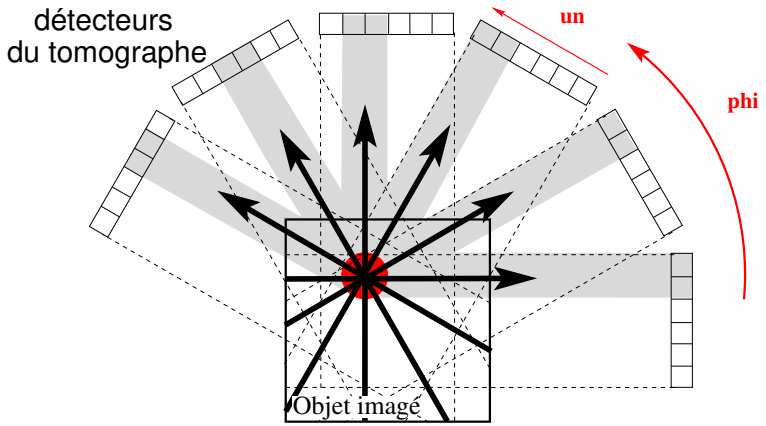
$\mathcal{P}/\mathcal{R}$: boucles naturelles de parallélisation

Opérateur de projection $\mathcal{P}$



$\mathcal{P}/\mathcal{R}$: boucles naturelles de parallélisation

Opérateur de projection $\mathcal{P}$



for un, phi **do**

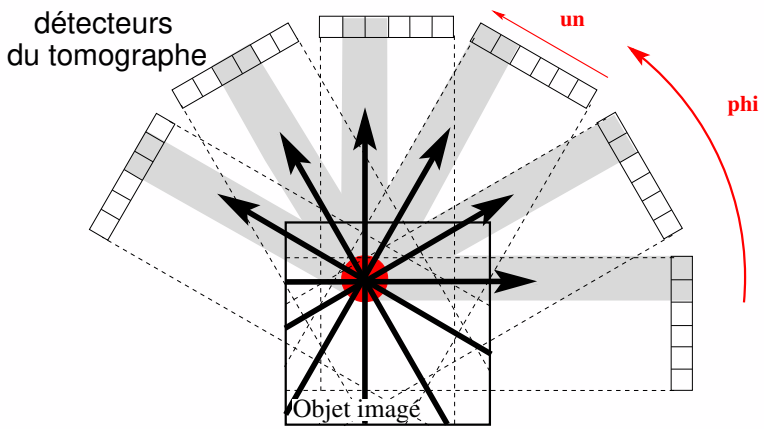
...

$\mathcal{P}$(un, phi)=...

end for

$\mathcal{P}/\mathcal{R}$: boucles naturelles de parallélisation

Opérateur de projection $\mathcal{P}$



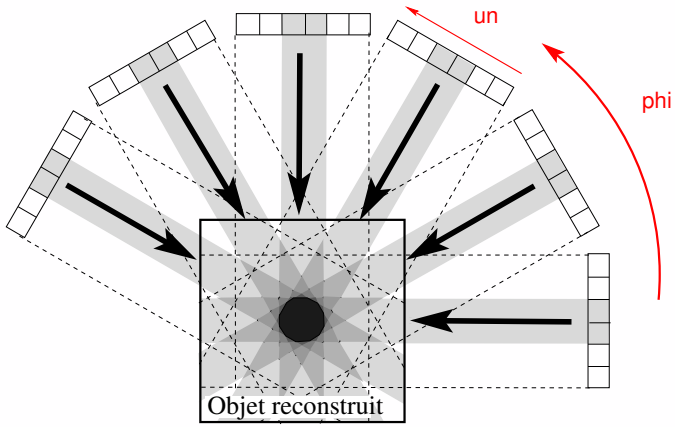
for `un, phi` **do**

...

$\mathcal{P}$(`un, phi`)=...

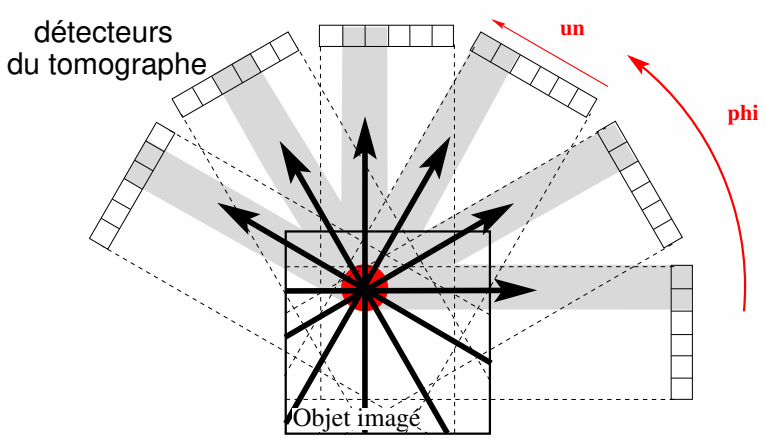
end for

Opérateur de rétroprojection $\mathcal{R}$



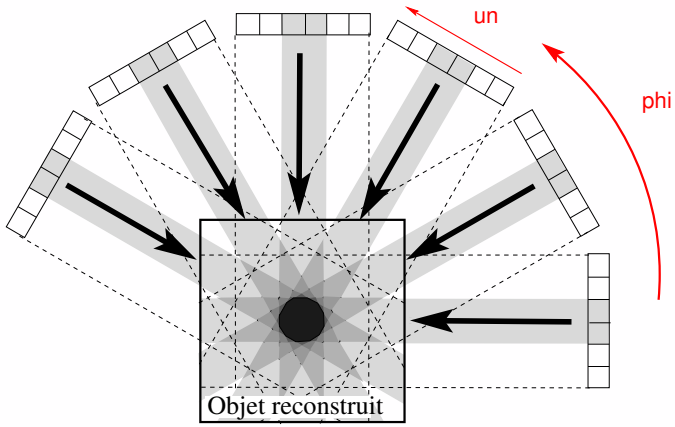
$\mathcal{P}/\mathcal{R}$: boucles naturelles de parallélisation

Opérateur de projection $\mathcal{P}$



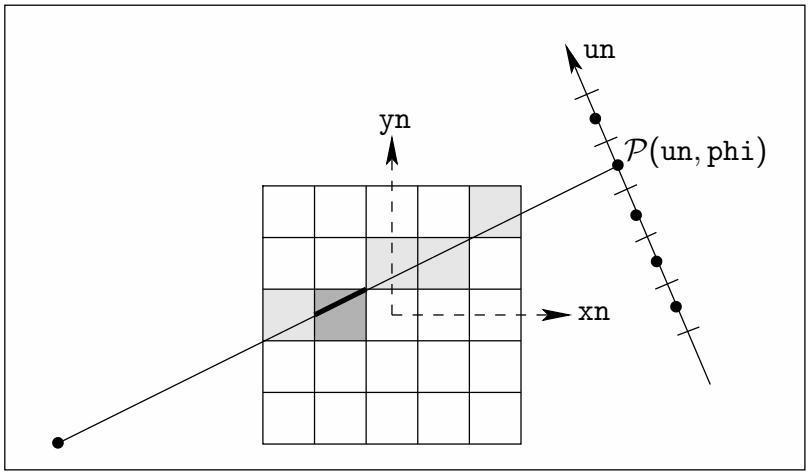
for un, phi **do**
  ...
  $\mathcal{P}$(un, phi)=...
end for

Opérateur de rétroprojection $\mathcal{R}$

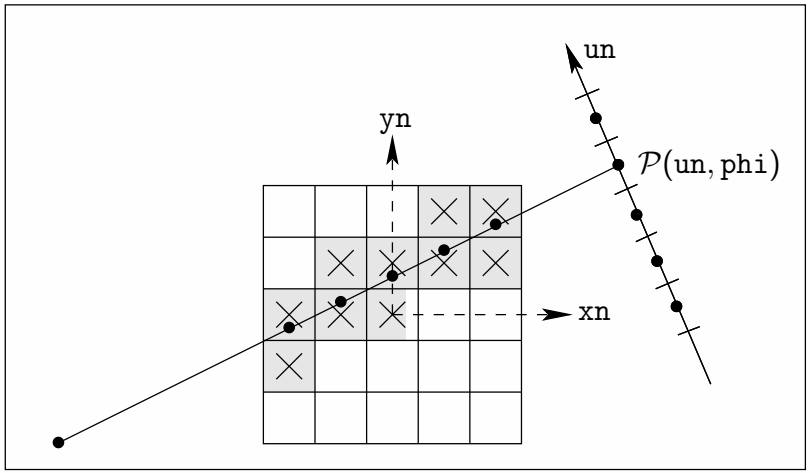


for xn, yn **do**
  ...
  $\mathcal{R}$(xn, yn)=...
end for

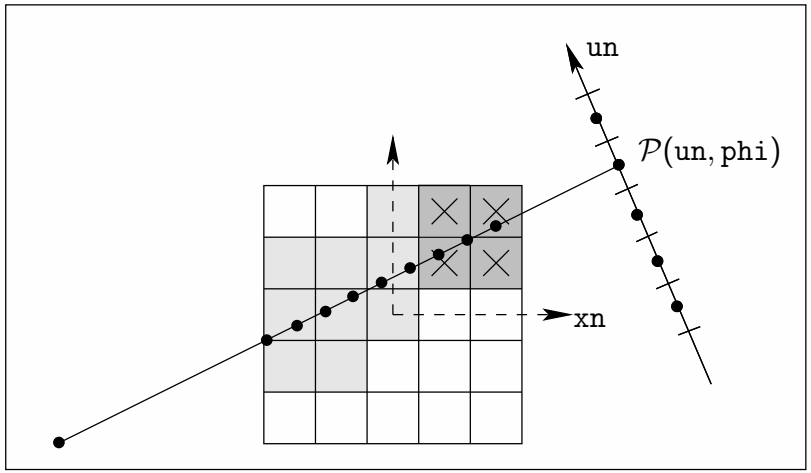
Modèle *ray-driven* $\mathcal{P}_{Siddon}$



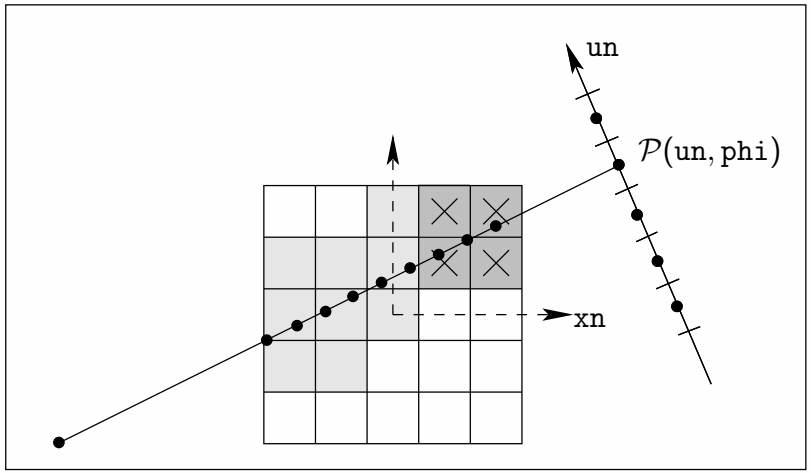
Modèle *ray-driven* $\mathcal{P}_{Joseph}$



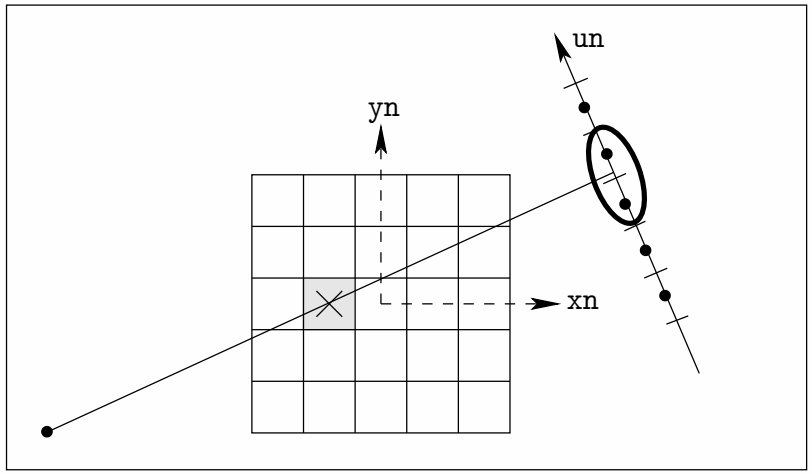
Modèle *ray-driven* $\mathcal{P}_{Regular}$



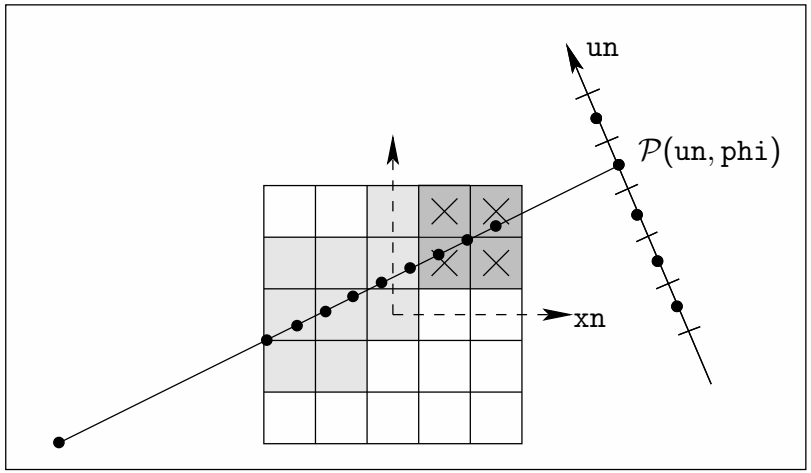
Modèle *ray-driven* $\mathcal{P}_{Regular}$



Modèle *Voxel-Driven* $\mathcal{R}_{VDL}$

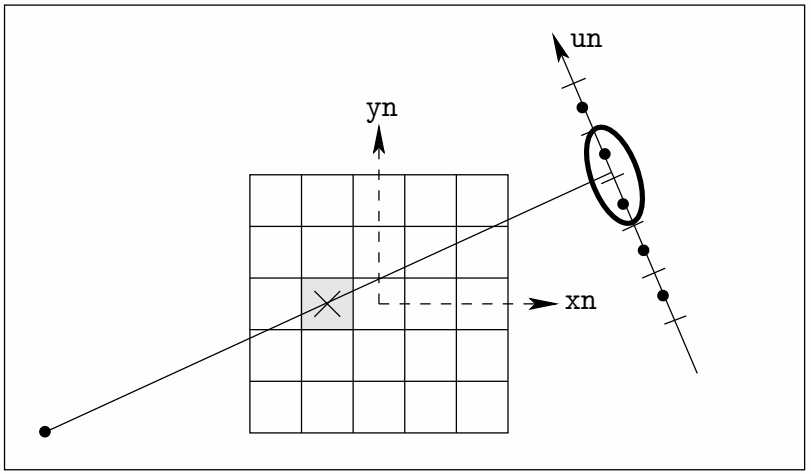


Modèle *ray-driven* $\mathcal{P}_{Regular}$



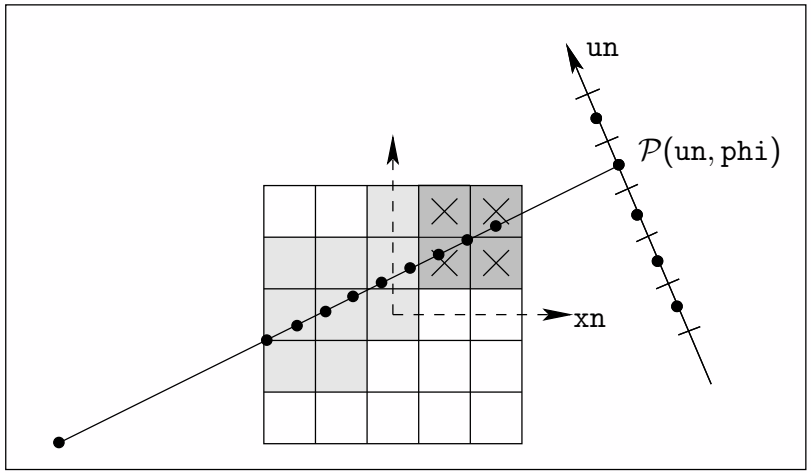
$$\mathcal{P}(\boldsymbol{f}) = \boldsymbol{H}_{\mathcal{P}} f$$

Modèle *Voxel-Driven* $\mathcal{R}_{VDL}$



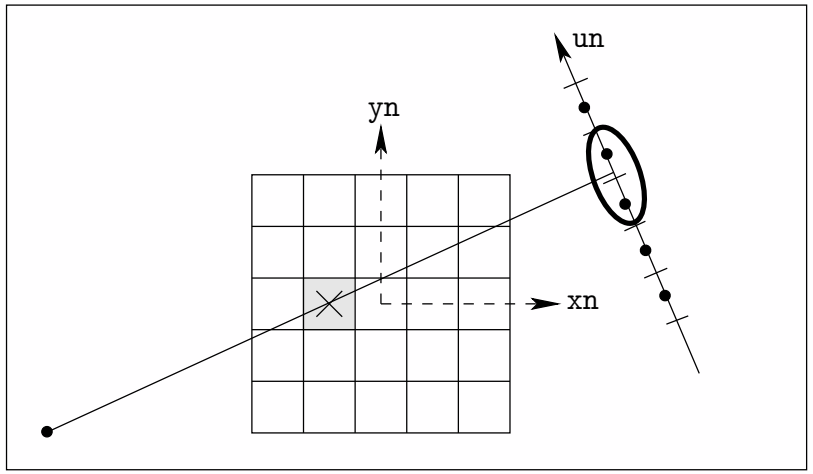
$$\mathcal{R}(\boldsymbol{g}) = \boldsymbol{H}_{\mathcal{R}}^{t} \boldsymbol{g}$$

Modèle *ray-driven* $\mathcal{P}_{Regular}$



$$\mathcal{P}(\boldsymbol{f}) = \boldsymbol{H}_{\mathcal{P}} f$$

Modèle *Voxel-Driven* $\mathcal{R}_{VDL}$

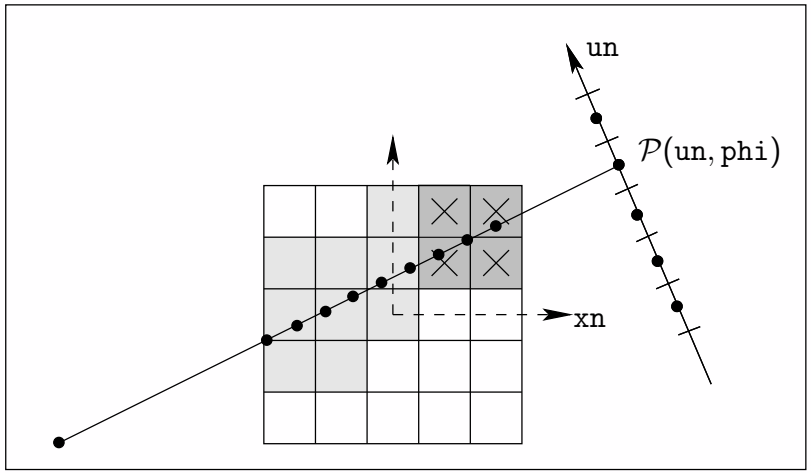


$$\mathcal{R}(\boldsymbol{g}) = \boldsymbol{H}_{\mathcal{R}}^{t} \boldsymbol{g}$$

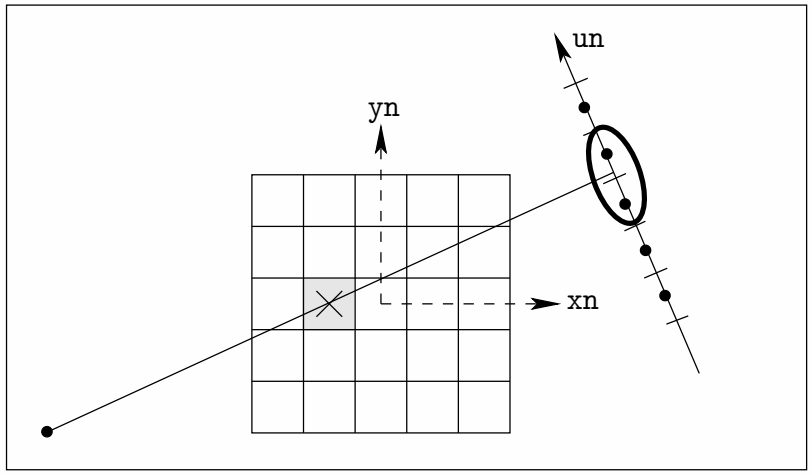
La paire $\mathcal{P}_{Regular}/\mathcal{R}_{VDL}$ est **non duale**

$\mathcal{P}/\mathcal{R}$ est une paire duale $\equiv$ $\forall \boldsymbol{f}, \forall \boldsymbol{g}, < \boldsymbol{g} \cdot \mathcal{P}(\boldsymbol{f}) > = < \mathcal{R}(\boldsymbol{g}) \cdot \boldsymbol{f} >$

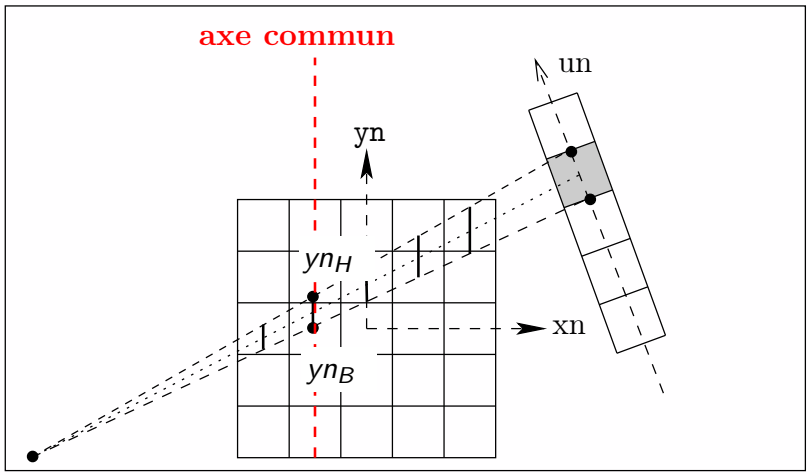
Modèle *ray-driven* $\mathcal{P}_{Regular}$



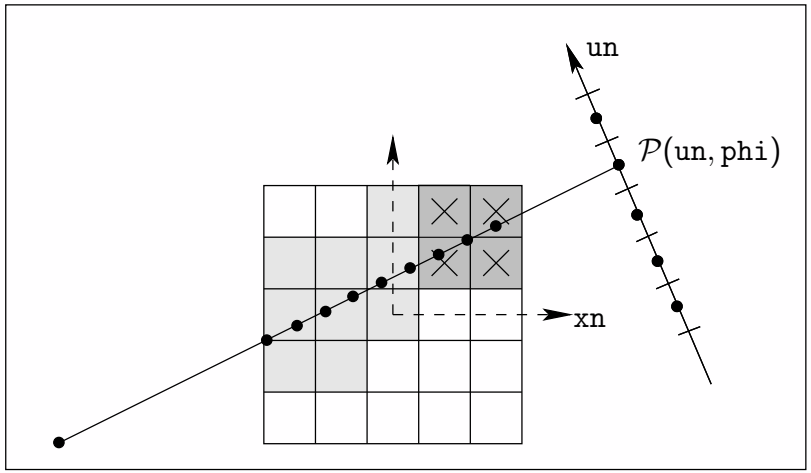
Modèle *Voxel-Driven* $\mathcal{R}_{VDL}$



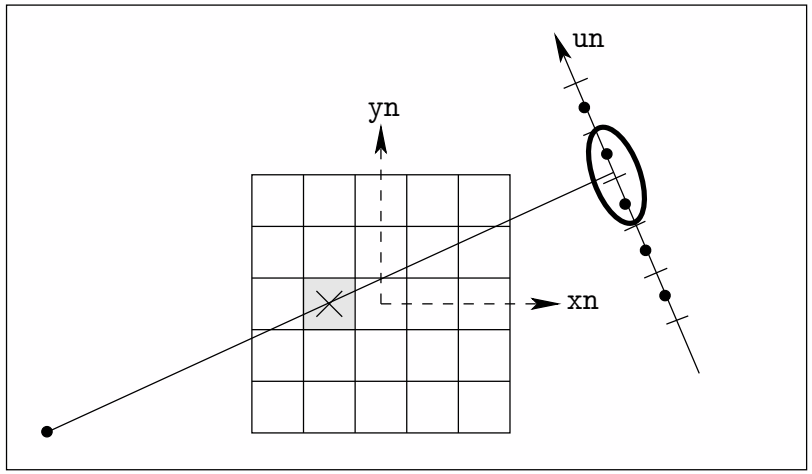
Modèle *Distance-Driven* $\mathcal{P}_{DD}$



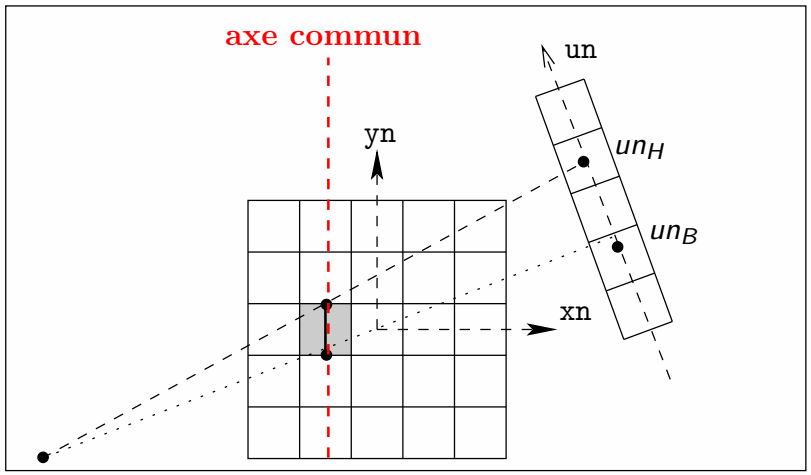
Modèle *ray-driven* $\mathcal{P}_{Regular}$



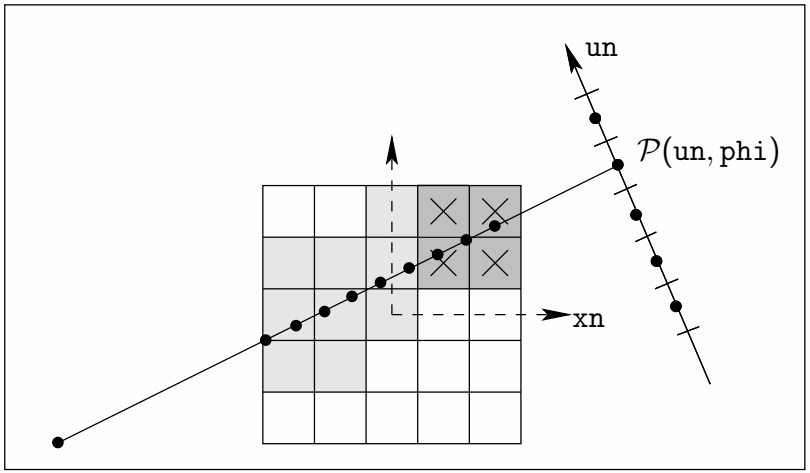
Modèle *Voxel-Driven* $\mathcal{R}_{VDL}$



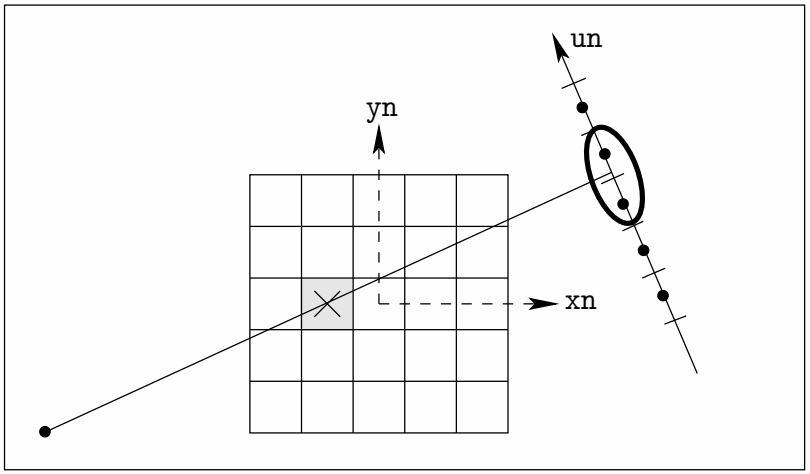
Modèle *Distance-Driven* $\mathcal{R}_{DD}$



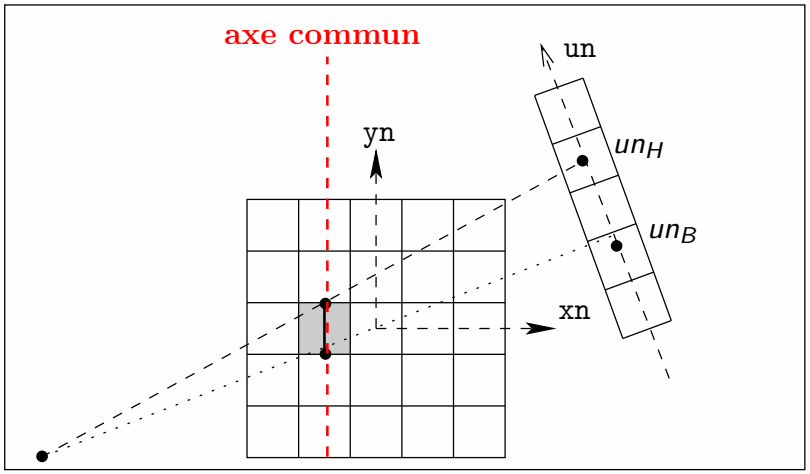
Modèle *ray-driven* $\mathcal{P}_{Regular}$



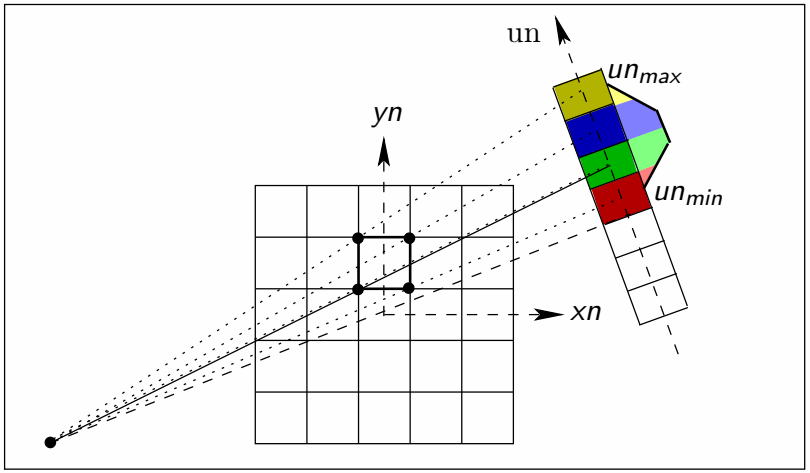
Modèle *Voxel-Driven* $\mathcal{R}_{VDL}$



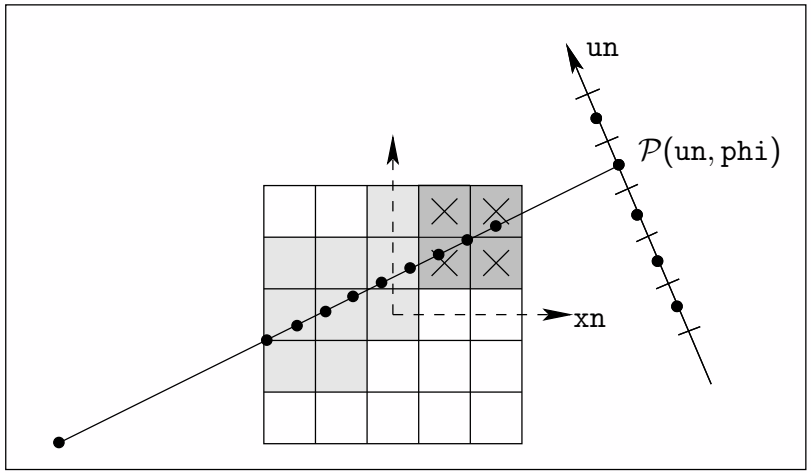
Modèle *Distance-Driven* $\mathcal{R}_{DD}$



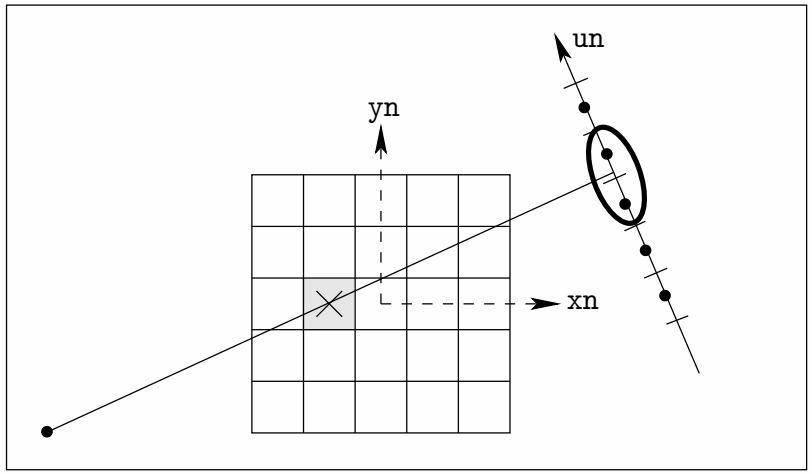
Modèle *Separable Footprint* (*transverse*)



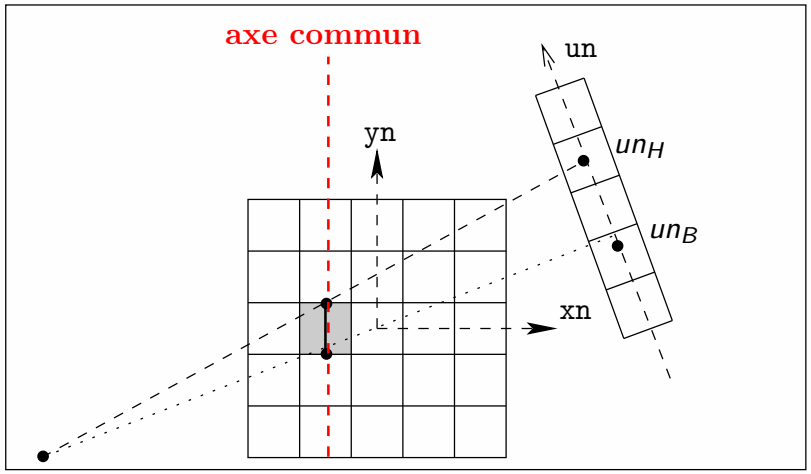
Modèle *ray-driven* $\mathcal{P}_{Regular}$



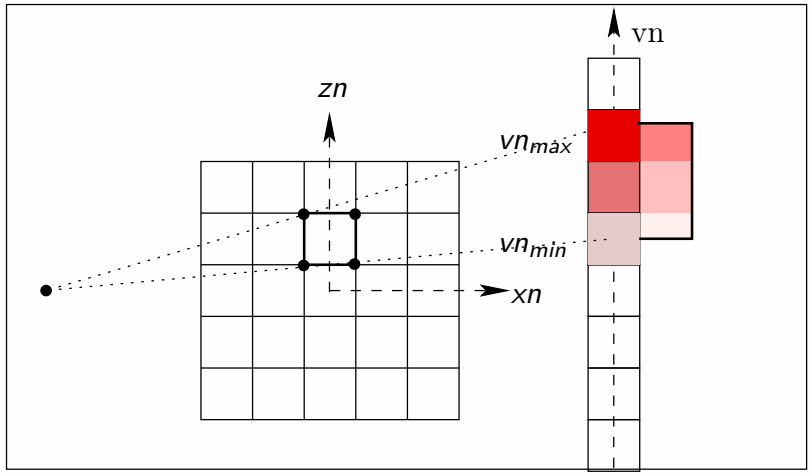
Modèle *Voxel-Driven* $\mathcal{R}_{VDL}$



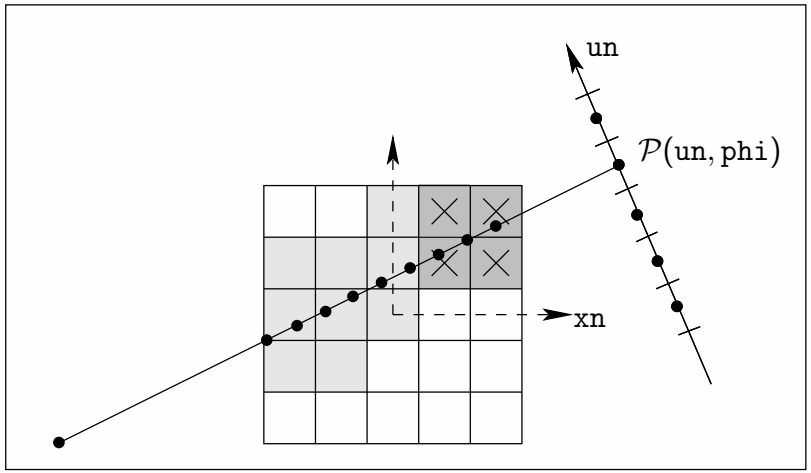
Modèle *Distance-Driven* $\mathcal{R}_{DD}$



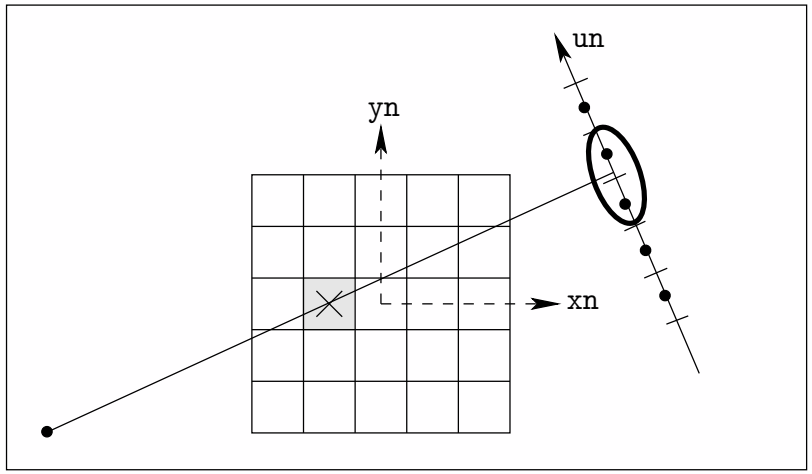
Modèle *Separable Footprint* (*axial*)



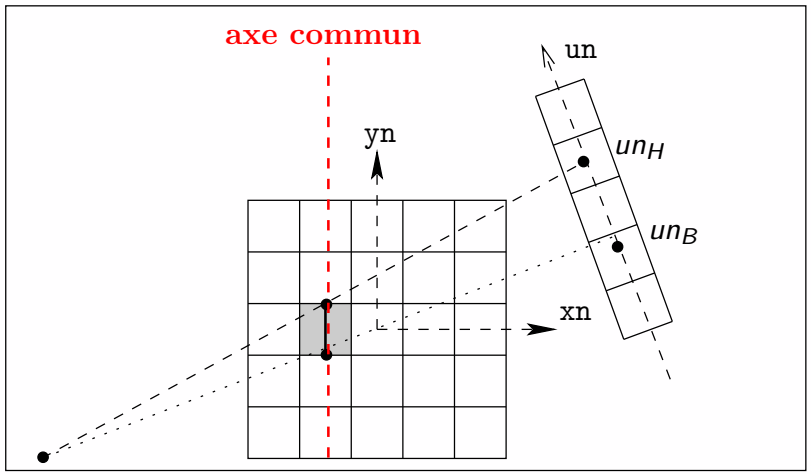
Modèle *ray-driven* $\mathcal{P}_{Regular}$



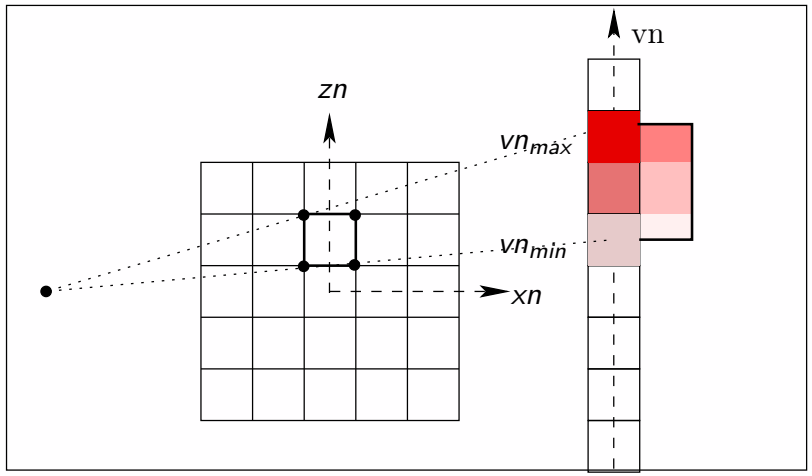
Modèle *Voxel-Driven* $\mathcal{R}_{VDL}$



Modèle *Distance-Driven* $\mathcal{R}_{DD}$



Modèle *Separable Footprint* (*axial*)



$\Longrightarrow$ $\mathcal{P}_{DD}/\mathcal{R}_{DD}$ et $\mathcal{P}_{SF}/\mathcal{R}_{SF}$ paires duales

Logiciel TomoBayes

TomoBayes

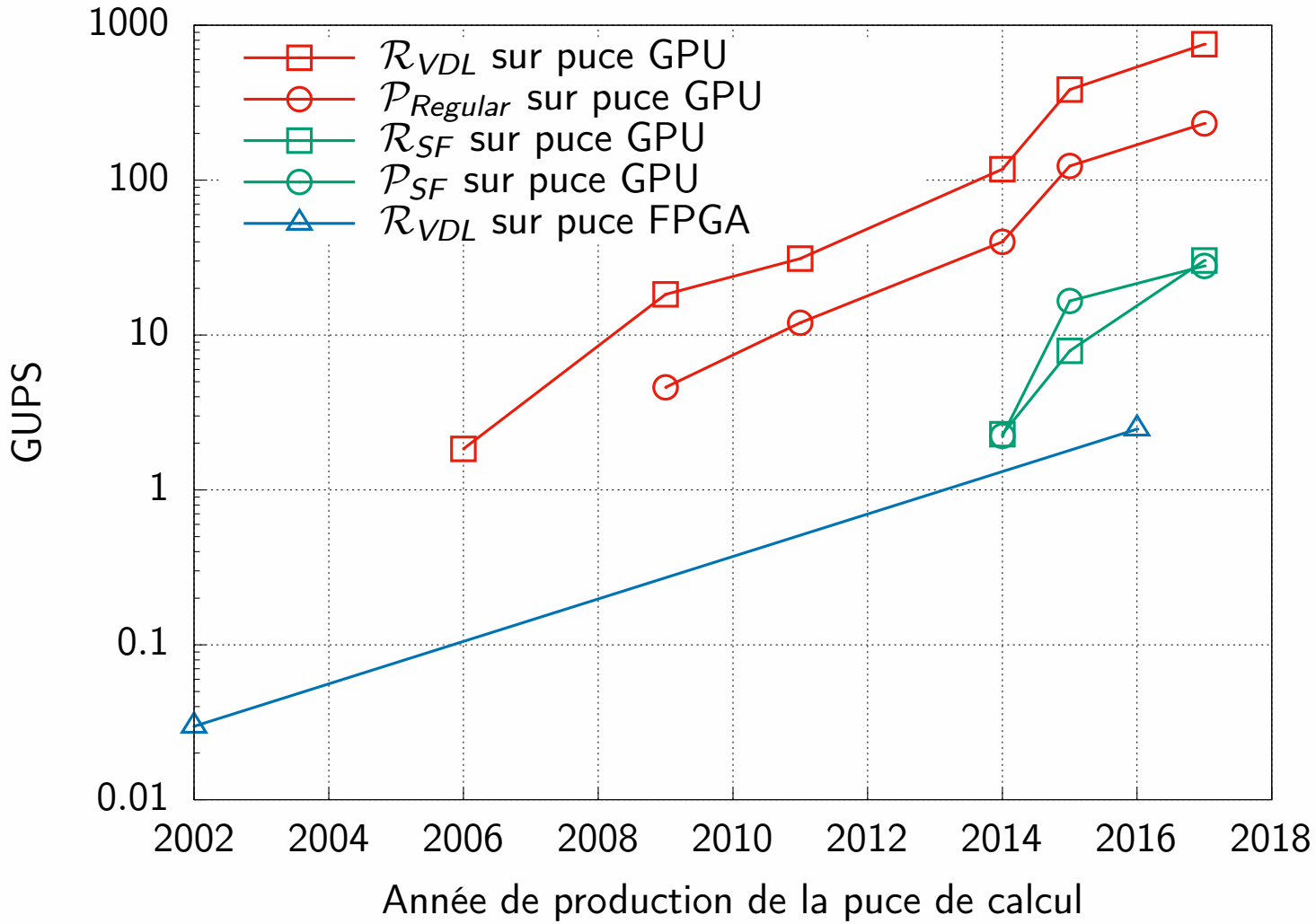
- co-propriété du L2S et de SAFRAN
- Algorithmes bayésiens
- Accélération sur GPU des opérateurs de projection et rétroprojection

Laptop démonstrateur avec boitiers externes FPGA et GPU



GPU versus FPGA

GUPS : Giga mises à jour de voxels ($\mathcal{R}$) ou de rayons ($\mathcal{P}$) par seconde

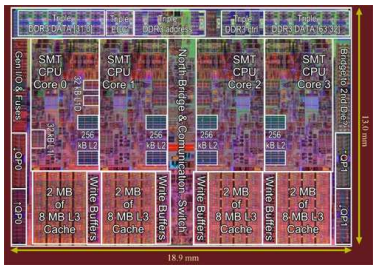


1. Adéquation Algorithme Architecture (A^3)
   - Problématique et positionnement
   - A^3 au sein du GPI
2. Calcul intensif sur GPU/FPGA
   - Paire de projection/rétroprojection $\mathcal{P}/\mathcal{R}$
3. GPU (Graphic Processing Units) : hardware and software
   - GPU (re)designed as a many core architecure
   - Programming in CUDA
   - A toy example : acceleration of matrix multiplication

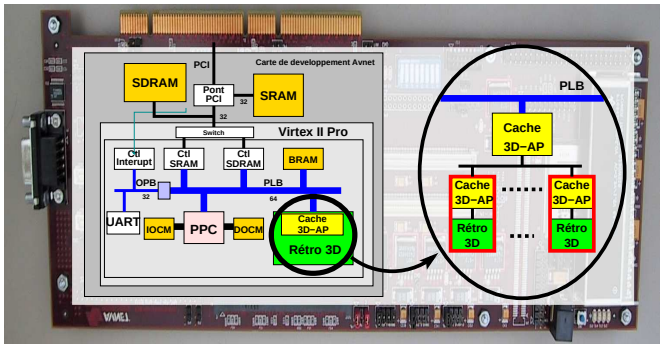
Calcul haute performance

High Performance Computing (HPC)

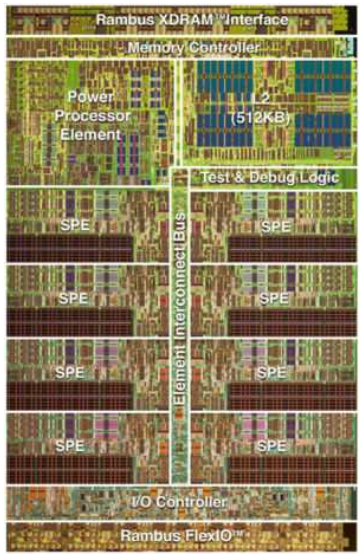
- Parallélisation sur machines multi-processeurs
  ➲ Efficace sur machine à mémoire distribuée
- Noeuds de calculs performants
  ➲ processeurs multi-core, many-core ou FPGA/ASIC



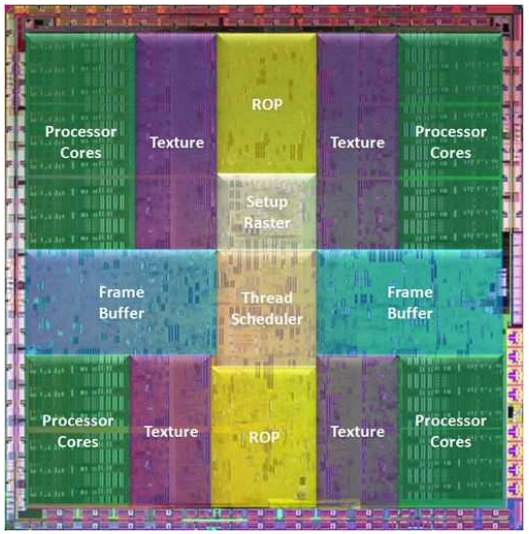
Intel Nehalem (4 coeurs)



SoPC (prototypage)



IBM Cell (8+1 coeurs)

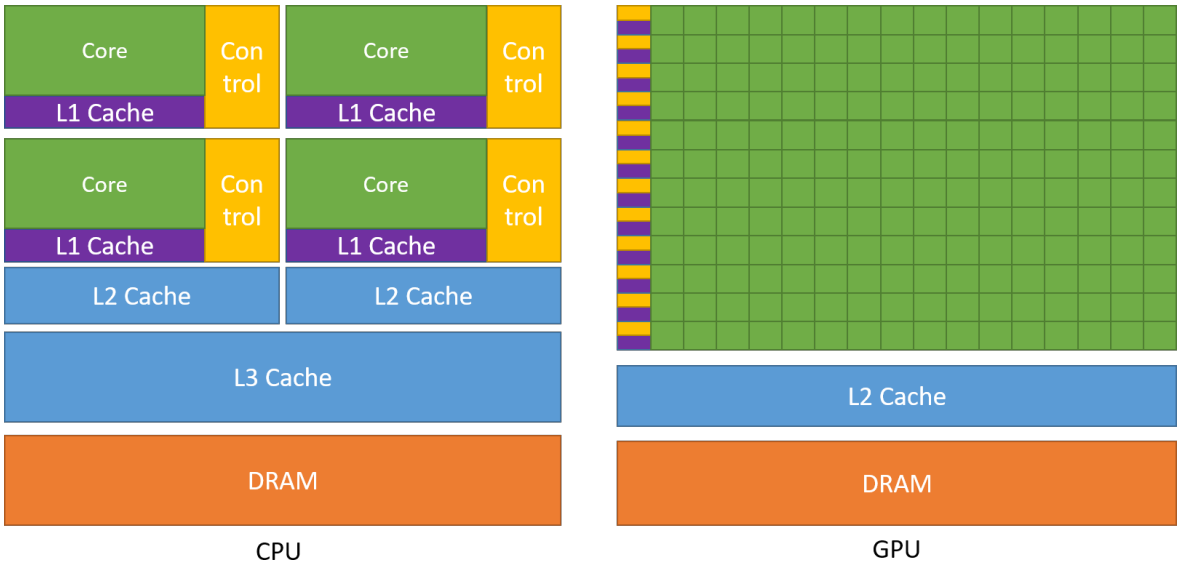


Nvidia GTX 200 (240 coeurs)

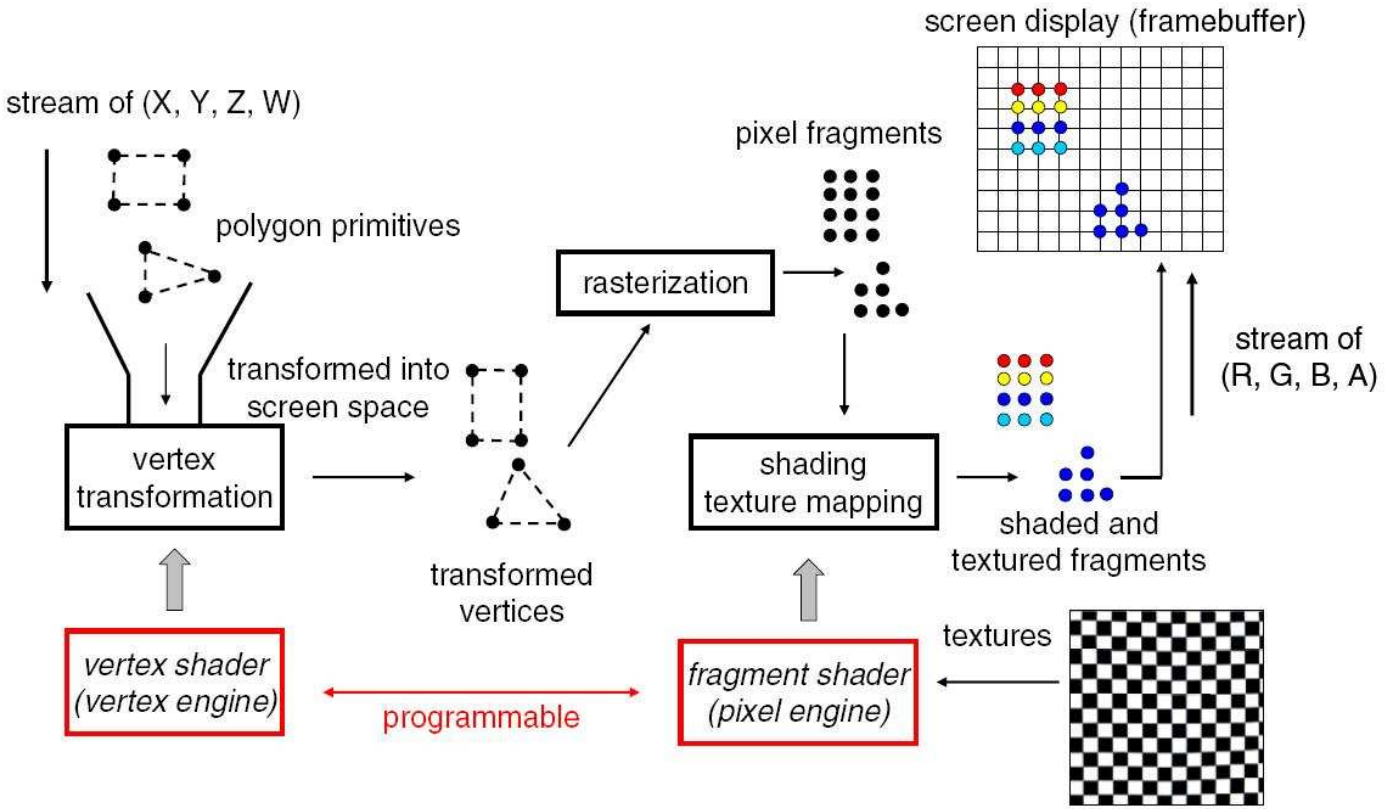
GPU : Graphic Processing Unit

Evolution vers une architecture *many core*

- A l'origine, architecture dédiée pour le rendu de volume
  $\Longrightarrow$ Pipeline graphique (prog. en OpenGL/Cg)
- Depuis 2006, architecture adaptée à la parallélisation de divers calculs scientifiques
  $\Longrightarrow$ CUDA : Common Unified Device Architecure (prog. en C)



Avant CUDA : pipeline graphique



Vertex Shader

Transformation géométrique

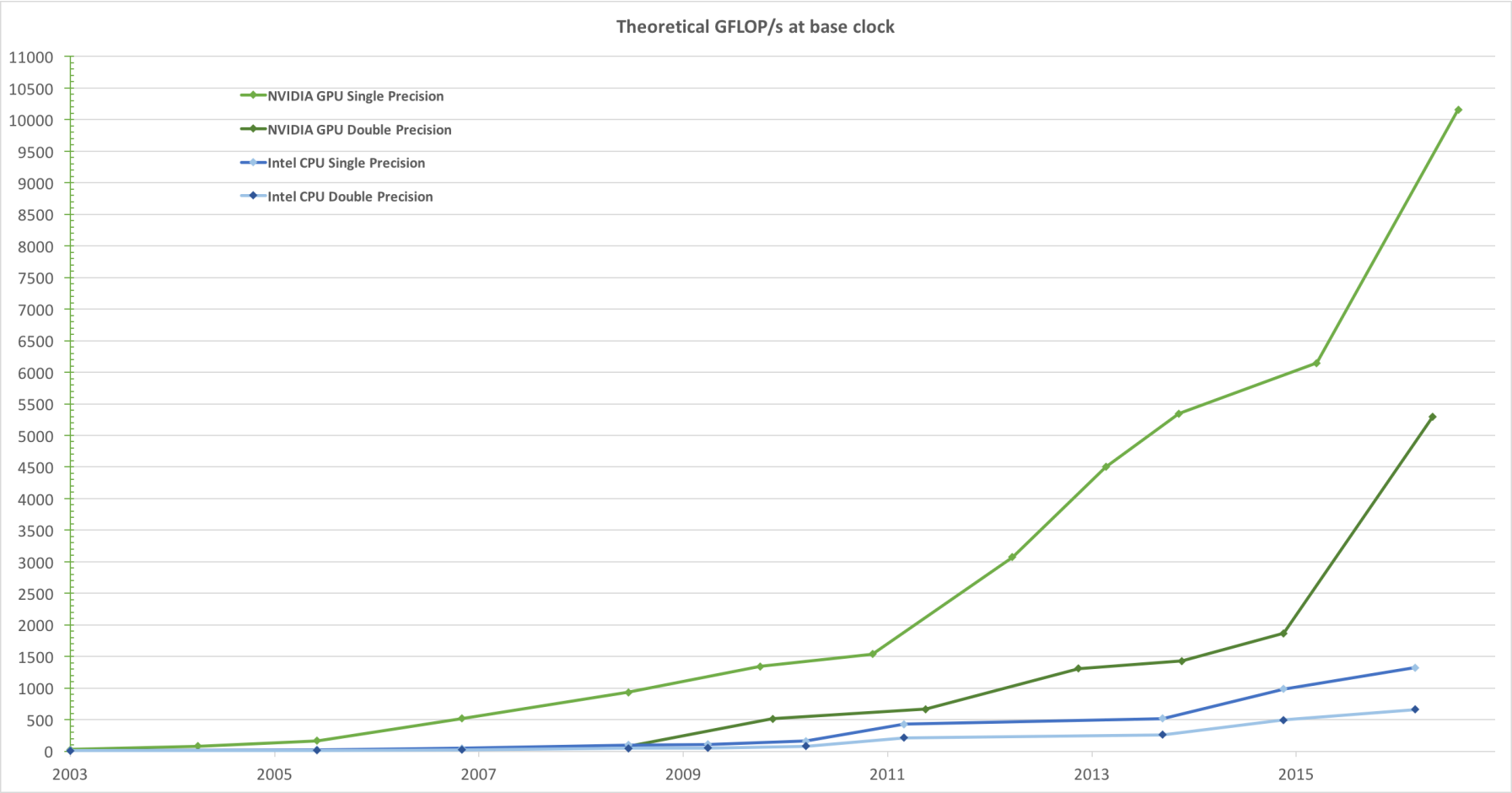
Rasterization

Polygon $->$ Fragments

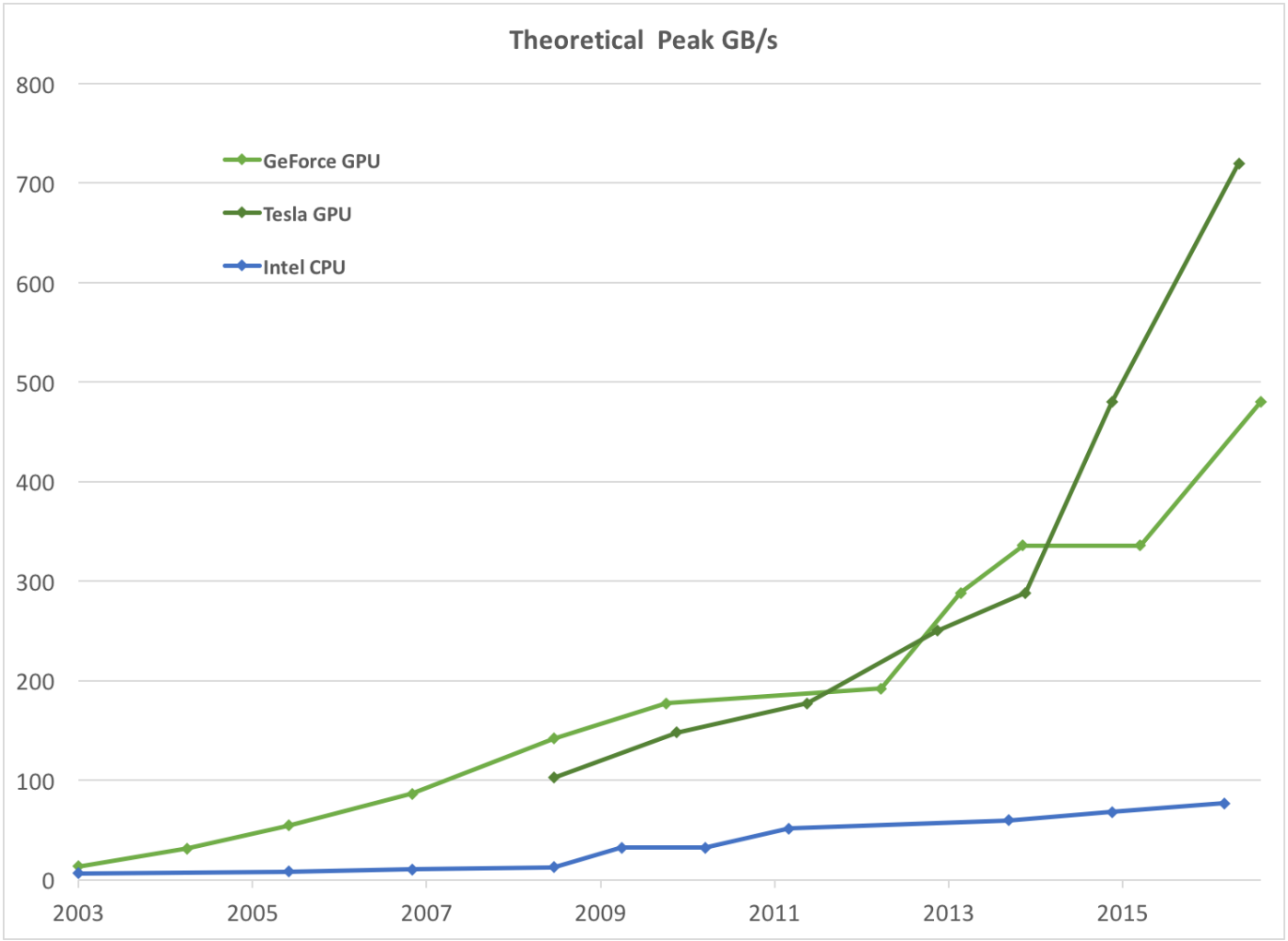
Fragment Shader

Calcul sur les Pixels

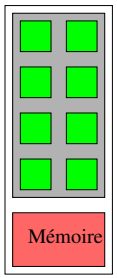
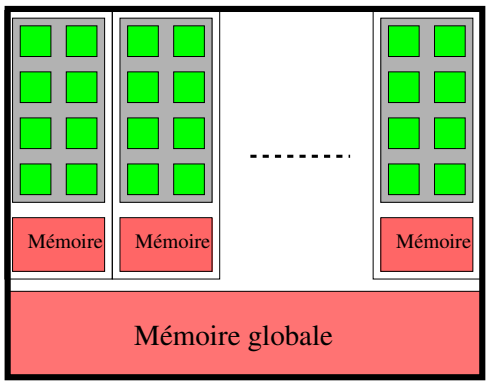
Puissance de calcul



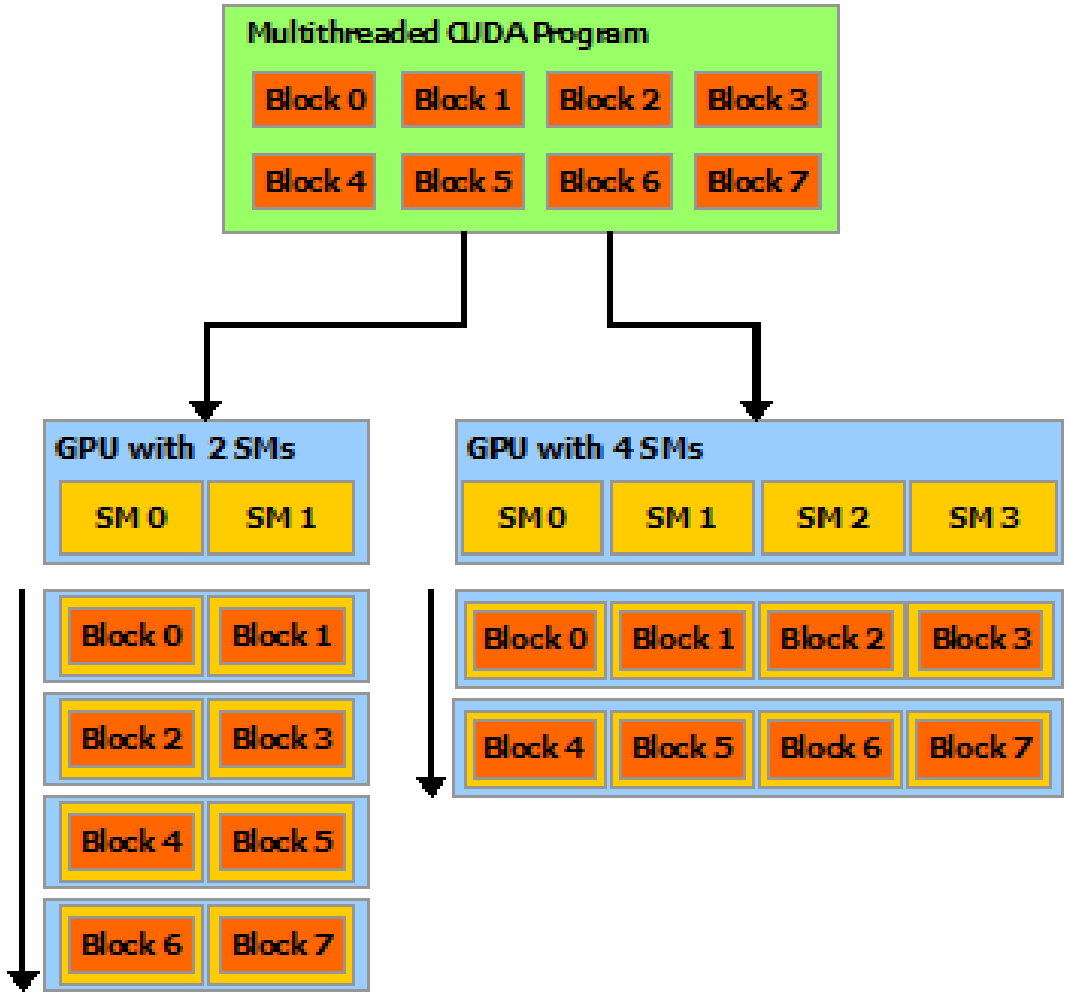
Débit mémoire



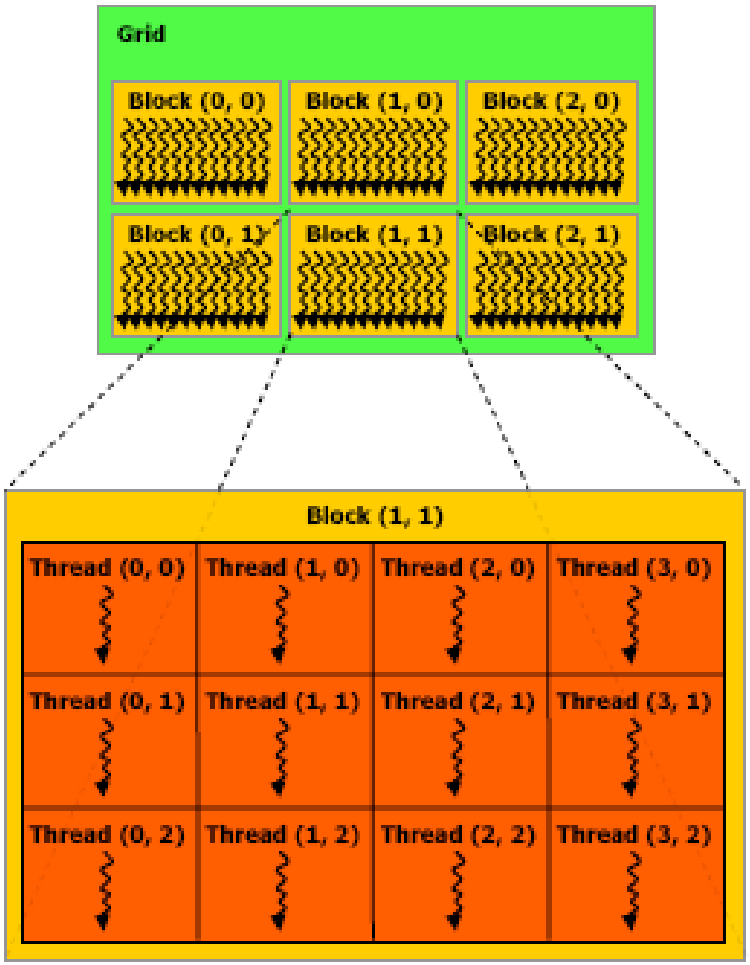
Découpage en threads

	Matériel	Logiciel	Exécution	
	un Streaming Processor (SP)	**un thread**	séquentielle	**(a)**
	un Streaming MultiProcessor (SM)	**un bloc de threads** (plusieurs warps)	parallèle (SIMT)	**(b)**
	une carte GPU (device)	**une grille de threads** (kernel)	parallèle (MIMT)	**(c)**

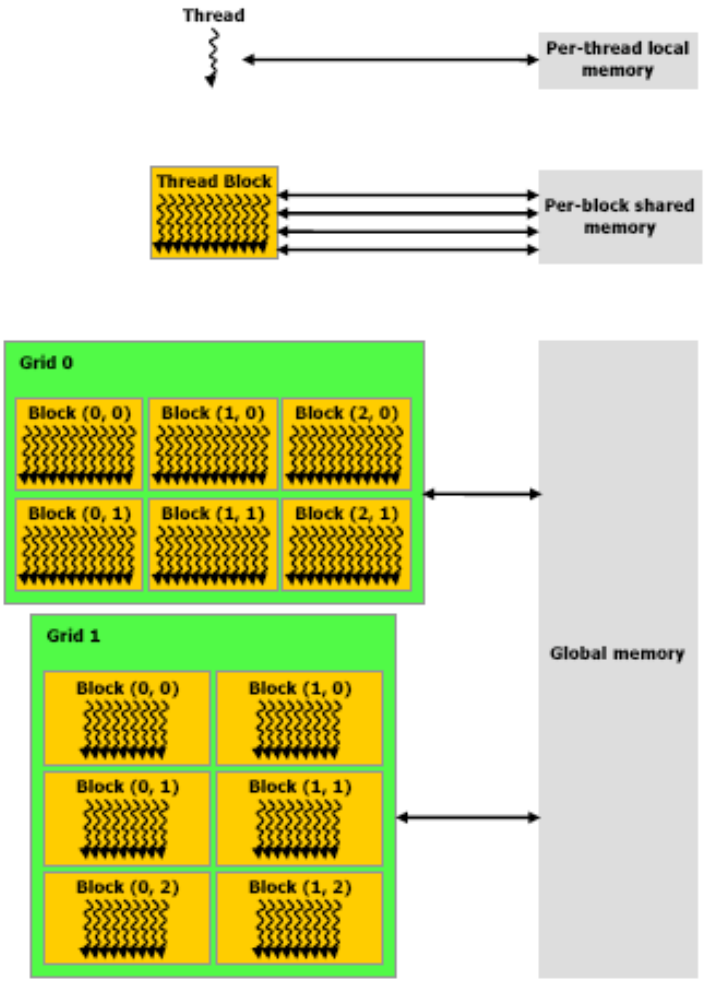
Ordonnancement des blocs de threads



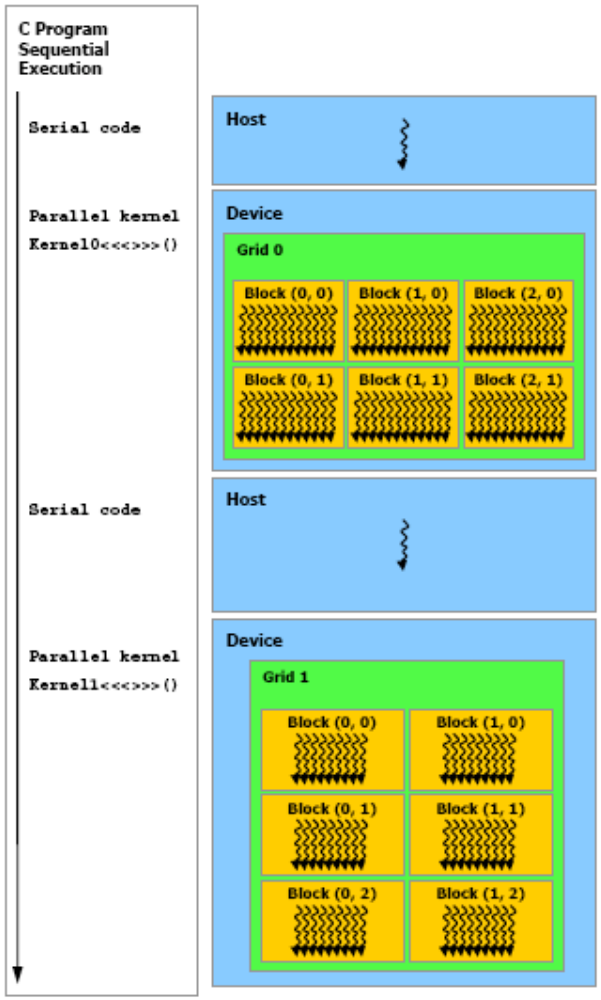
Un **id** par thread et un **id** par bloc de threads



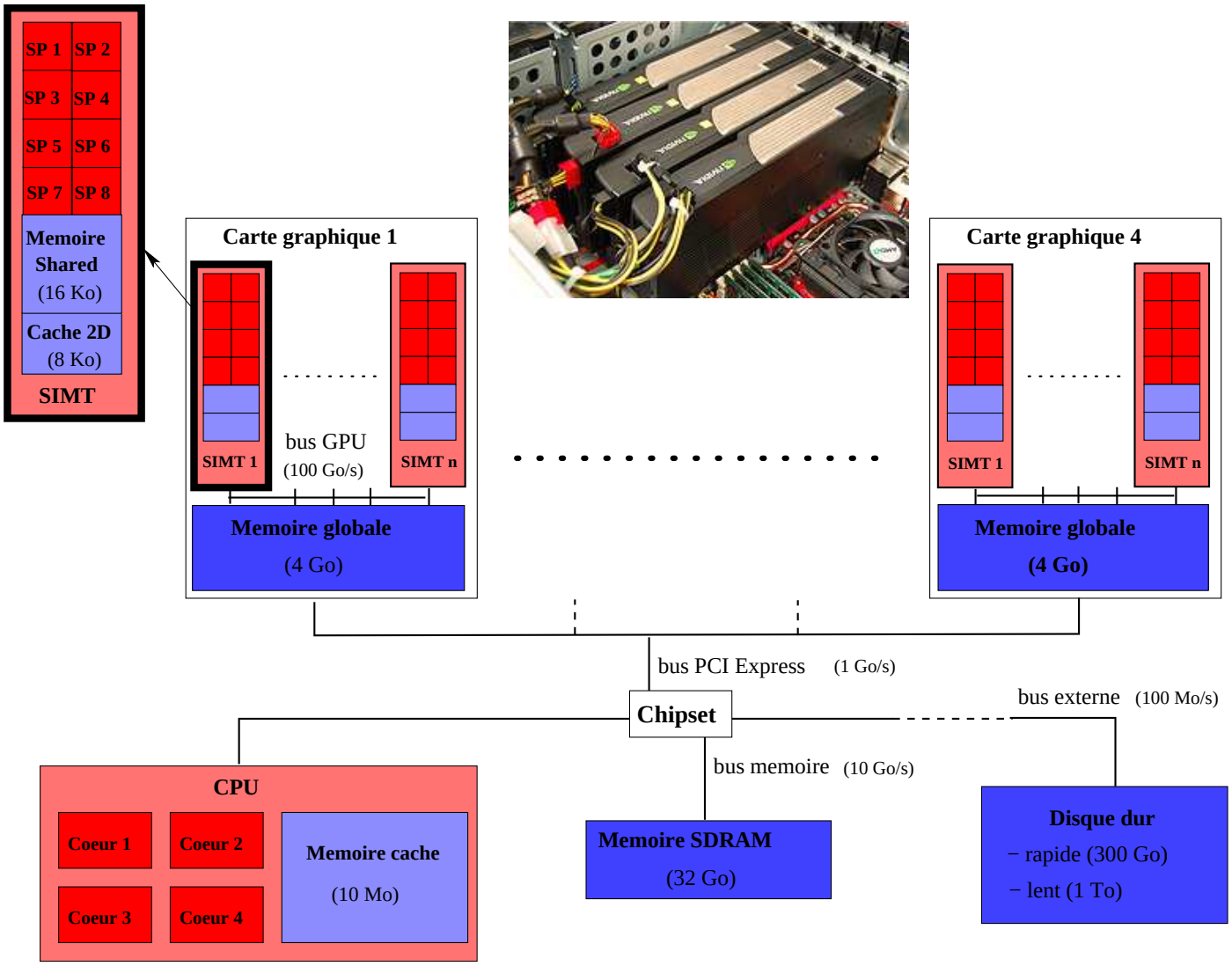
Hiérarchie mémoire



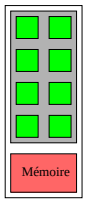
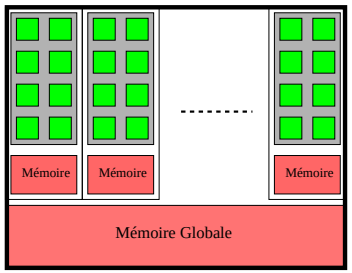
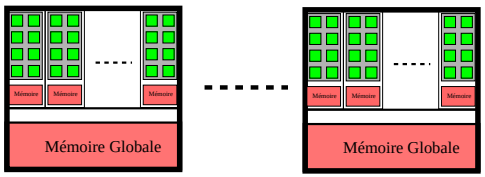
PC hote et carte graphique



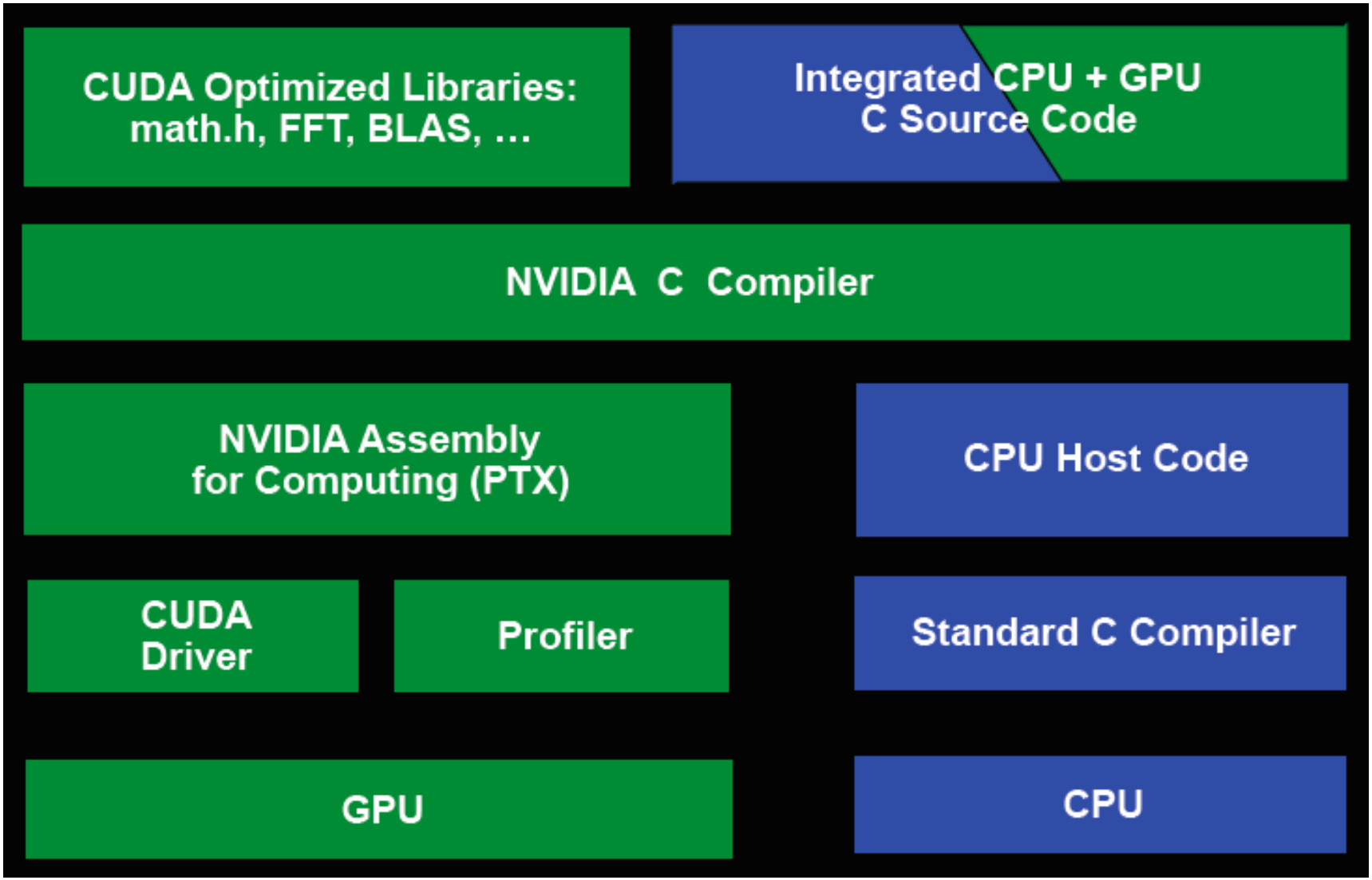
Supercalculateur personnel



Découpage en threads et en **grilles (kernels)**

	Matériel	Logiciel	Exécution	
	un Stream Processor (SP)	un thread	séquentielle	**(a)**
	un processeur SIMT	un bloc de threads	parallèle (SIMT)	**(b)**
	une carte GPU (device)	une grille de threads (kernel)	parallèle (MIMD) mémoire centralisée	**(c)**
	PC multi−carte	threads du PC hote via librairie pthread (un thread CPU = un kernel GPU)	parallèle (MIMD) mémoire distribuée	**(d)**

Flot de développement logiciel



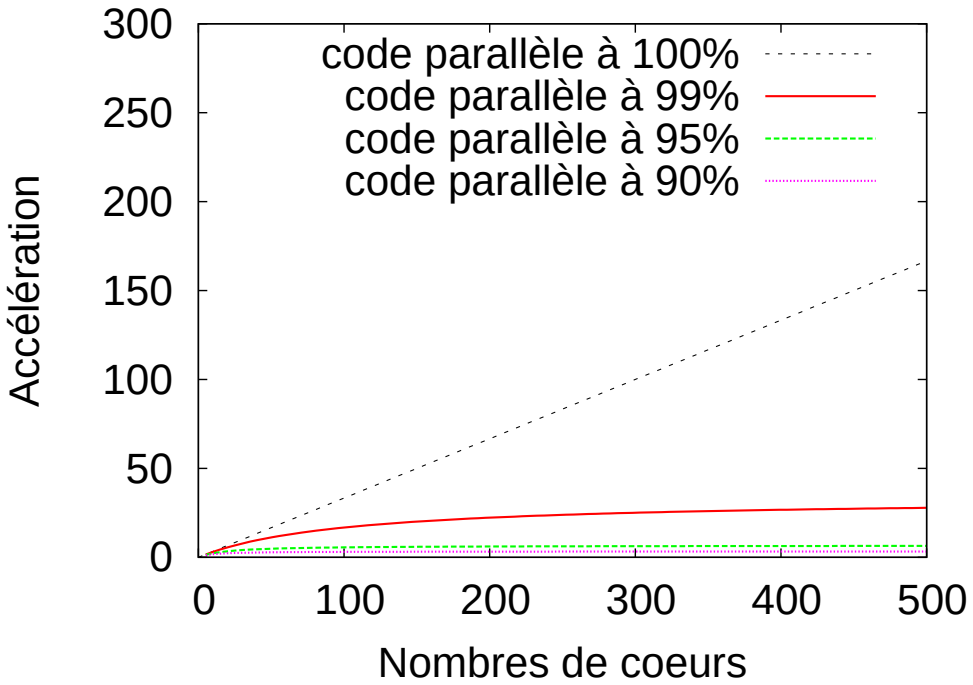
Programmation GPU

1) Parallélisation de l'algorithme

➲ nourrir en threads (plus ou moins indépendants) le GPU

n coeurs (1 Ghz) vs 1 coeur (3 Ghz)

taux de parallélisation	accélération GTX 200 (240 coeurs)
100 %	80
99 %	24
95 %	6
90 %	3



Programmation GPU

1) Parallélisation de l'algorithme

➲ nourrir en threads (plus ou moins indépendants) le GPU

2) Implémentation GPU

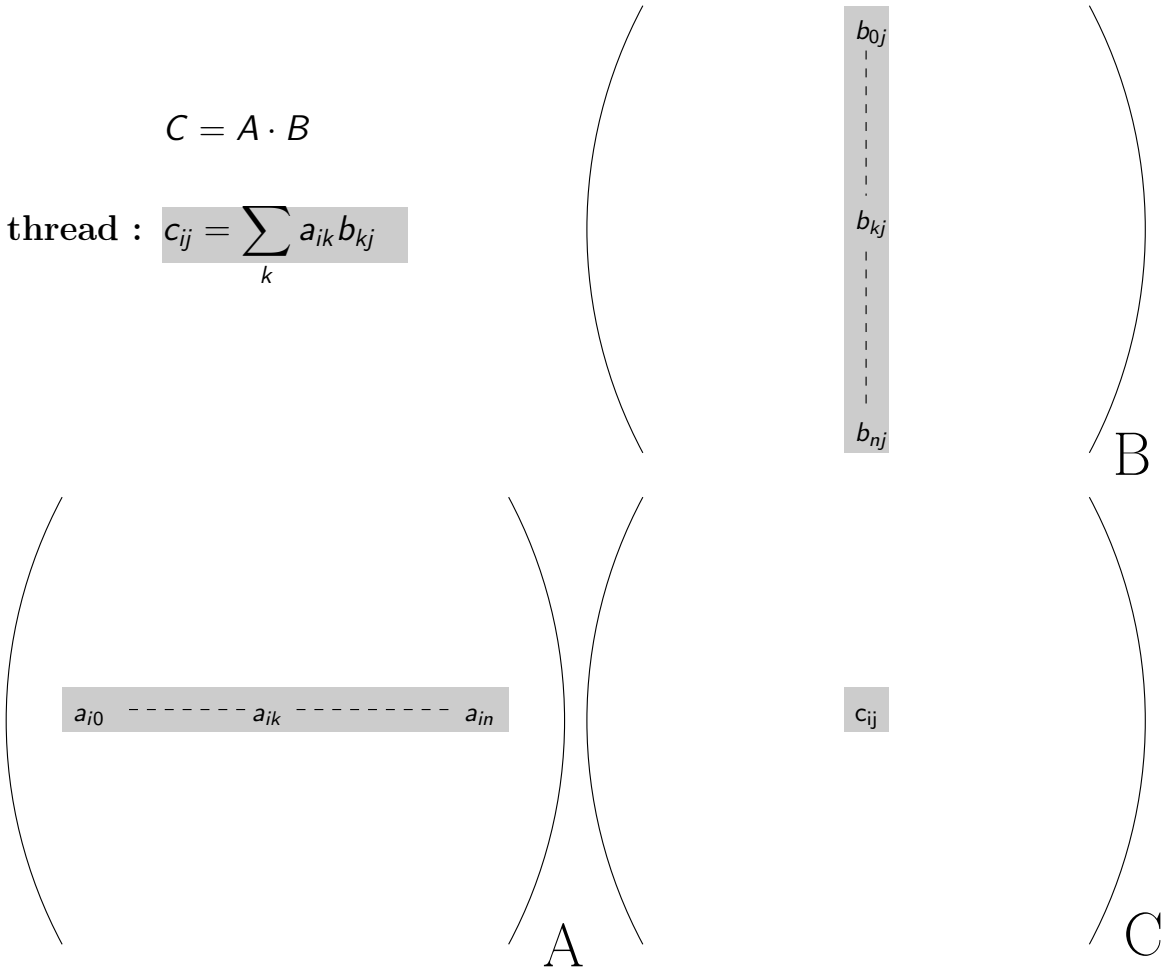
Selon l'intensité arithmétique du code (puissance de calcul exploitée / débit des données), l'execution sera soit *memory bound* soit *computation bound* (ex : calcul X^k [?])

➲ optimisation du code portera alors soit sur les **accès mémoire** ou soit sur la **complexité arithmétique**

Parallélisation du calcul matriciel

$$C = A \cdot B$$

un thread : $c_{ij} = \sum_k a_{ik} b_{kj}$

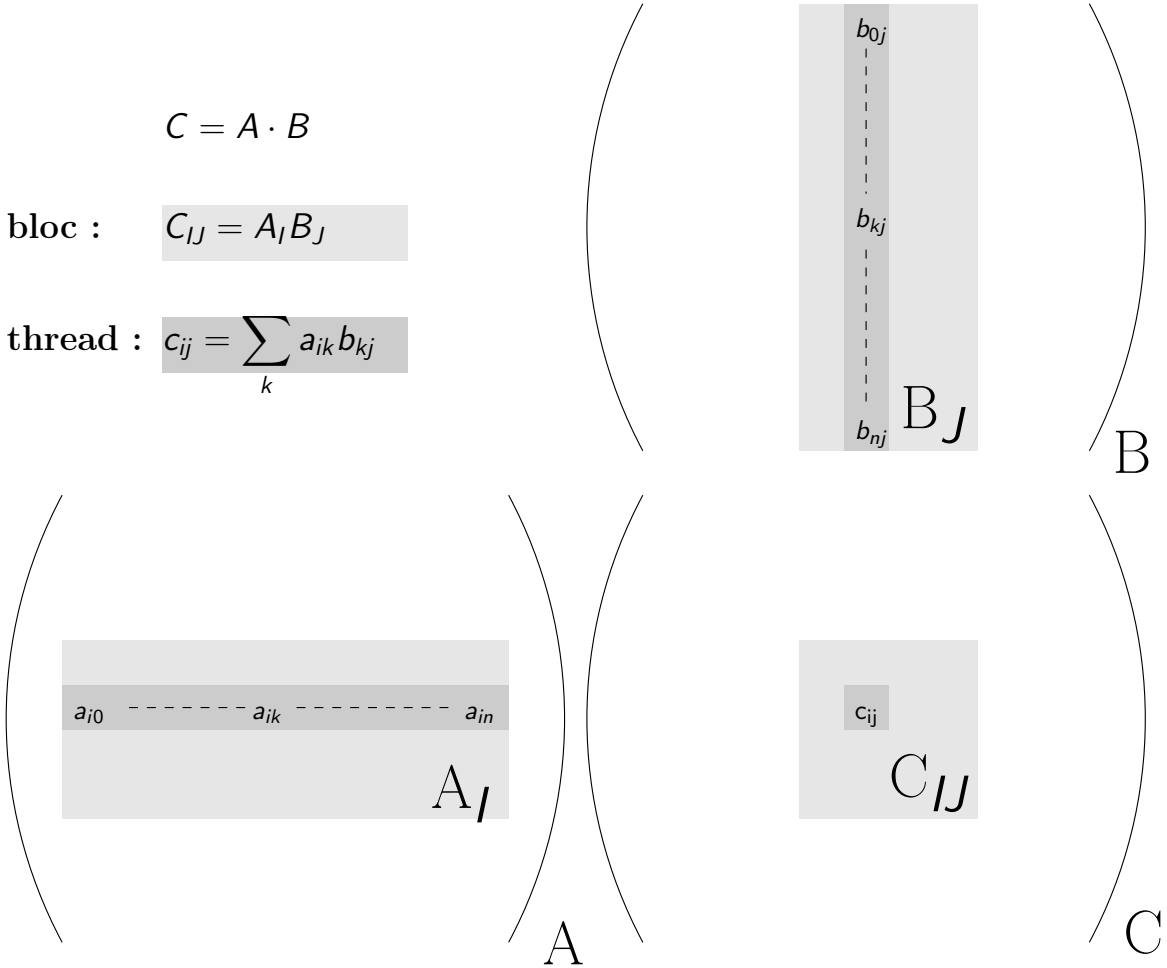


Découpage en blocs de threads

$$C = A \cdot B$$

un bloc : $C_{IJ} = A_I B_J$

un thread : $c_{ij} = \sum_k a_{ik} b_{kj}$



kernel = code des threads executés sur le GPU

```
__global__ void matrixMul_kernel( float* C, float* A, float* B,int matrix_size) {
float Csum ;
int ifirst ,jfirst ;
int i,j ;

ifirst =blockIdx.x*BLOCK_SIZE ;
jfirst =blockIdx.y*BLOCK_SIZE ;

i=ifirst +threadIdx.x ;
j=jfirst +threadIdx.y ;

for (k = 0 ; k < matrix_size ; k++)
   Csum += A[i][k] * B[k][j] ;

C[i][j] = Csum ;

}
```

Lancement du kernel depuis le PC hôte

```
#define BLOCK_SIZE 16

void matrixMul_host(int N) {

...

//setup execution parameters
dim3 threads(BLOCK_SIZE, BLOCK_SIZE);
dim3 grid(N /BLOCK_SIZE , N /BLOCK_SIZE );

//execute the kernel
matrixMul_kernel<<< grid, threads >>>(C_device, A_device, B_device, N);

...

}
```

Gestion de la mémoire GPU via le PC hôte

```
#define BLOCK_SIZE 16

void matrixMul_host(int N) {

// allocate host memory int mem_size=N^2*sizeof(float);
float* A_host = (float*) malloc(mem_size);
float* B_host = (float*) malloc(mem_size);
float* C_host = (float*) malloc(mem_size);

// allocate device memory
float* A_device,B_device,C_device;
cudaMalloc((void**) &A_device, mem_size);
cudaMalloc((void**) &B_device, mem_size);
cudaMalloc((void**) &C_device, mem_size);

// copy host memory to device cudaMemcpy(A_device, A_host, mem_size,cudaMemcpyHostToDevice);
cudaMemcpy(B_device, B_host, mem_size,cudaMemcpyHostToDevice);

//setup execution parameters
dim3 threads(BLOCK_SIZE, BLOCK_SIZE);
dim3 grid(N /BLOCK_SIZE , N /BLOCK_SIZE );

// execute the kernel
matrixMul_kernel<<< grid, threads >>>(C_device, A_device, B_device, N);

// copy result from device to host
cudaMemcpy(C_host, C_device, mem_size,cudaMemcpyDeviceToHost);

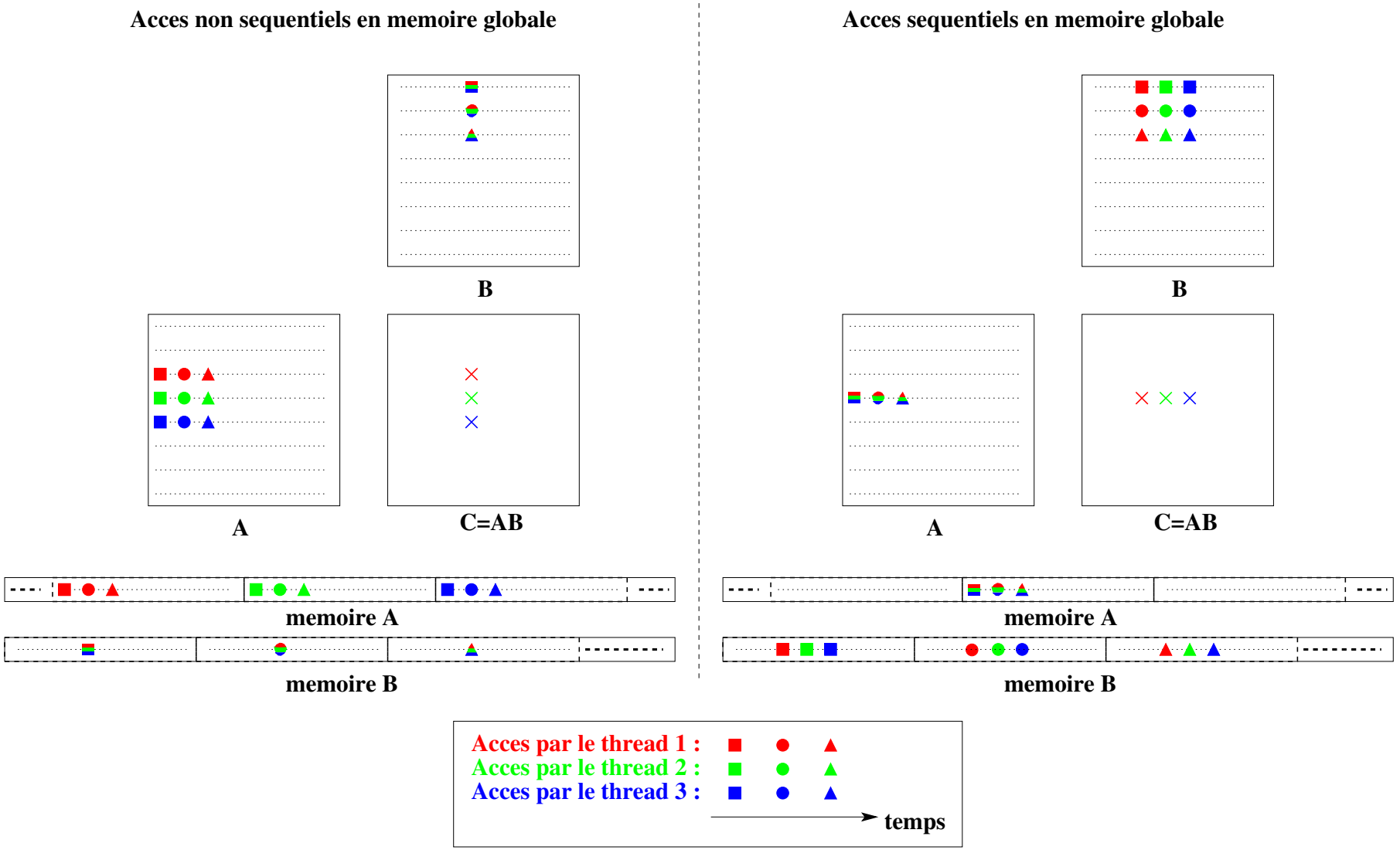
}
```

Temps GPU

Matrices de taille 1024·1024

	Processeur	Temps d'exécution	Transfert mémoire
C non optimisé	Xeon Quad core 2.7 Ghz	9.35 s	
Cuda	Testla C1060 240 PE @1,3 Ghz	1.35 s **(*6,9)**	$< 1\%$

Accès séquentiels à la mémoire globale



Temps GPU avec accès mémoire sequentiels

Matrices de taille 1024·1024

	Processeur	Temps d'exécution	Transfert mémoire
C non optimisé	Xeon Quad core 2.7 Ghz	9.35 s	
Cuda	Testla C1060 240 PE @1,3 Ghz	1.35 s **(*6,9)**	$< 1\%$
Cuda acces seq.	Testla C1060 240 PE @1,3 Ghz	124 m s **(*10,9)**	5%

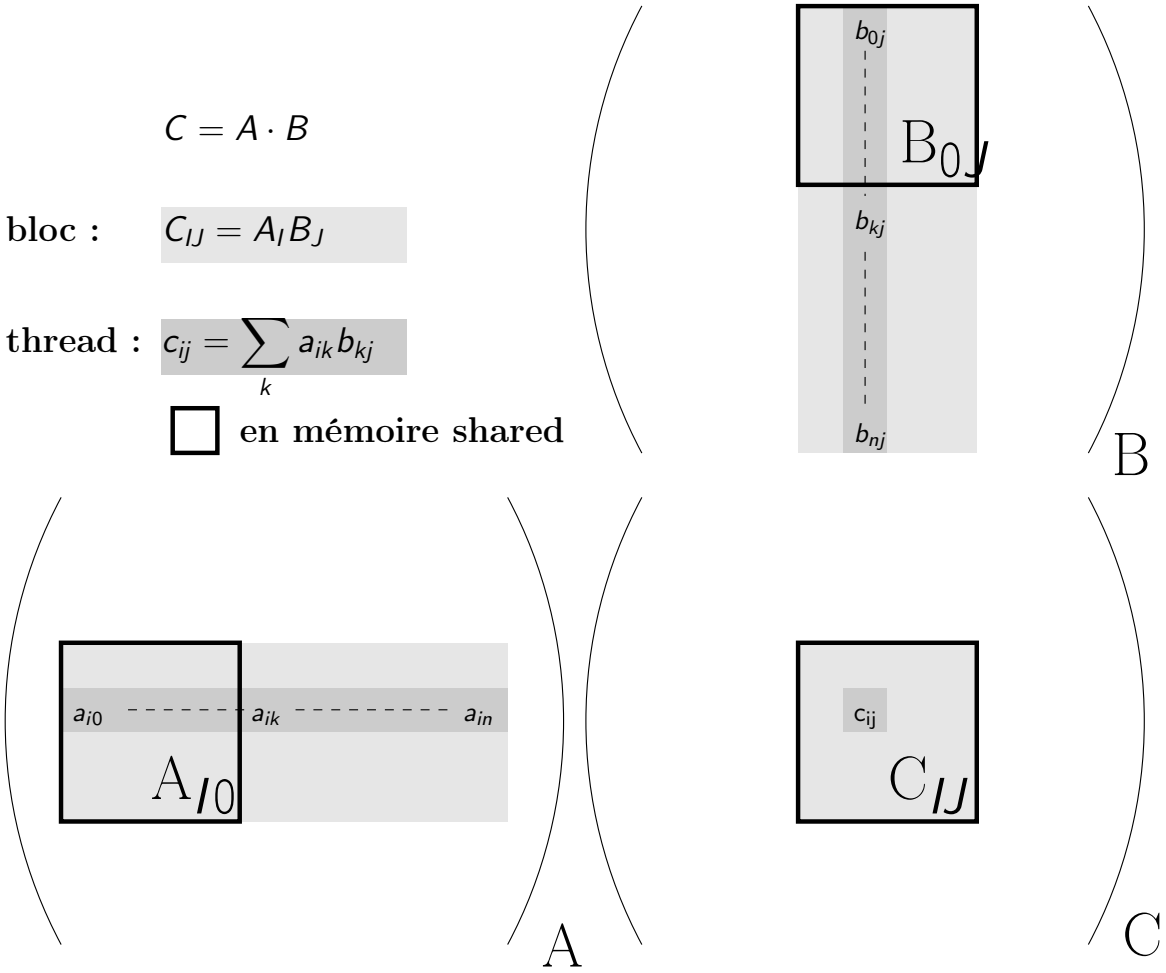
Optimisation des accès mémoire

$C = A \cdot B$

un bloc : $C_{IJ} = A_I B_J$

un thread : $c_{ij} = \sum_k a_{ik} b_{kj}$

en mémoire shared



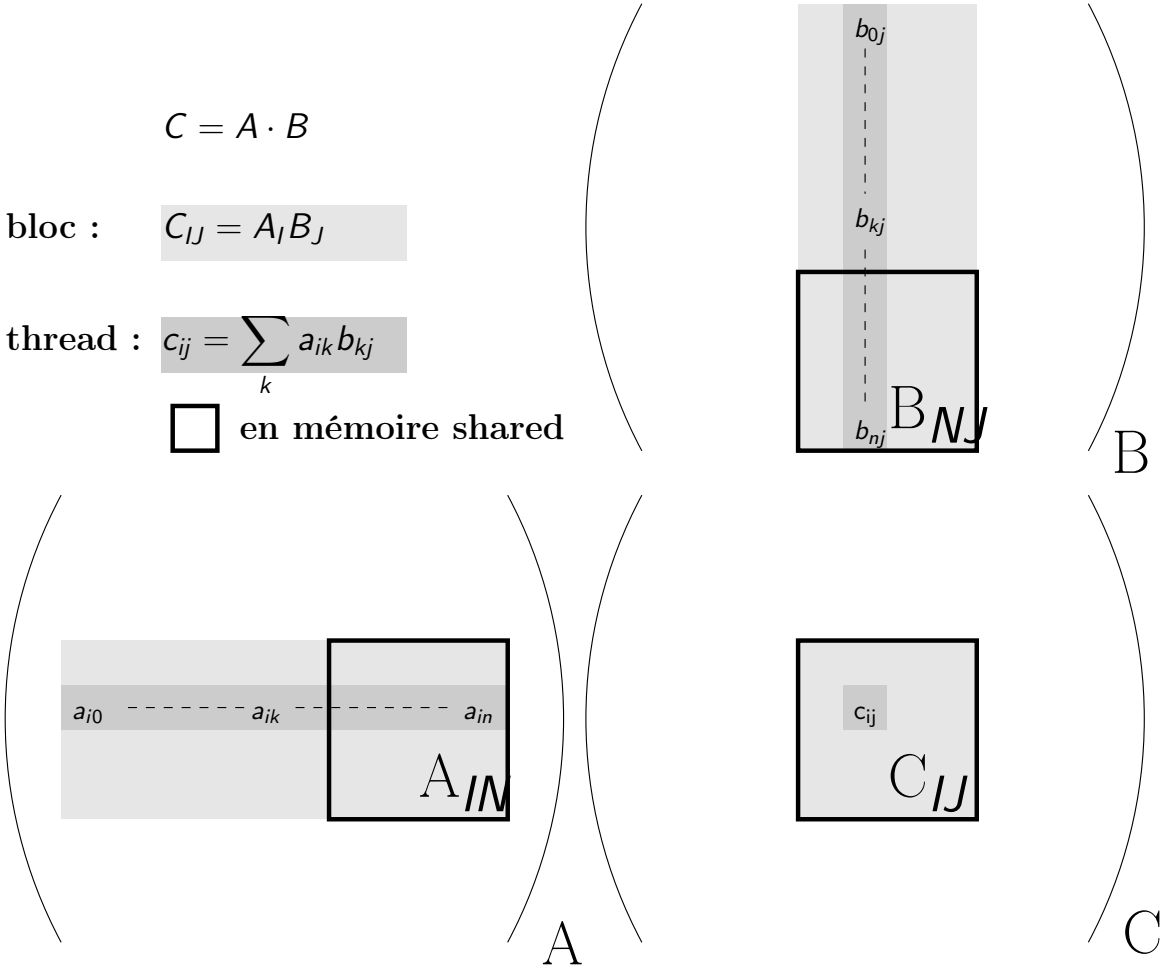
Optimisation des accès mémoire

$C = A \cdot B$

un bloc : $C_{IJ} = A_I B_J$

un thread : $c_{ij} = \sum_k a_{ik} b_{kj}$

en mémoire shared



Variable type qualifiers

__device__

- en mémoire globale
- durée de vie de l'application
- accessible par tous les threads de la grille et par le hôte via la librairie runtime

__constant__

- en mémoire globale (accès via cache constante)
- durée de vie de l'application
- accessible par tous les threads de la grille et par le hôte via la librairie runtime

__shared__

- en mémoire shared (locale à un coeur SIMT)
- durée de vie du bloc de threads
- **seulement accessible par les threads d'un même bloc**

Temps GPU optimisé

Matrices de taille 1024·1024

	Processeur	Temps d'exécution	Transfert mémoire
C non optimisé	Xeon Quad core 2.7 Ghz	9.35 s	
Cuda	Testla C1060 240 PE @1,3 Ghz	1.35 s **(*6,9)**	$< 1\%$
Cuda acces seq.	Testla C1060 240 PE @1,3 Ghz	124 m s **(*10,9)**	5%
Cuda shared mem.	Testla C1060 240 PE @1,3 Ghz	17,5 m s **(*7,1)**	34%

Librairie CUBLAS : CUda Basic Linear Algebra Subprograms

```
#include <cublas.h>
#include <cutil.h>

int main(void) {
float alpha = 1.0f, beta = 0.0f;
int N = 1024;
int mem_size = 1024*1024*sizeof(float);

// Allocate host memory
float* A_host = (float*) malloc(mem_size);
float* B_host = (float*) malloc(mem_size);
float* C_host = (float*) malloc(mem_size);

cublasInit();

//Allocate device memory
float* A_device,B_device,C_device;
cublasAlloc(N*N, sizeof(float), (void **)&A_device);
cublasAlloc(N*N, sizeof(float), (void **)&B_device);
cublasAlloc(N*N, sizeof(float), (void **)&C_device);

// copy host memory to device
cublasSetMatrix(N,N, sizeof(float), A_host, N, A_device, N);
cublasSetMatrix(N,N, sizeof(float), B_host, N, B_device, N);

//Calcul matriciel sur le GPU
cublasSgemm('n', 'n', N, N, N, alpha, A_device, N,B_device, N, beta, C_device, N);

//Récupération du résultat sur le PC hôte
cublasGetMatrix(N,N, sizeof(float), C_device,N, C_host, N);
}
```

Temps CUBLAS

Matrices de taille 1024·1024

	Processeur	Temps d'exécution	Transfert mémoire
C non optimisé	Xeon Quad core 2.7 Ghz	9.35 s	
Cuda	Testla C1060 240 PE @1,3 Ghz	1.35 s **(*6,9)**	$< 1\%$
Cuda acces seq.	Testla C1060 240 PE @1,3 Ghz	124 m s **(*10,9)**	5%
Cuda shared mem.	Testla C1060 240 PE @1,3 Ghz	17,5 m s **(*7,1)**	34%
CUBLAS	Testla C1060 240 PE @1,3 Ghz	12,8 m s **(*1,4)**	43%